



AKADEMIA E FORCAVE TË ARMATOSURA
FAKULTETI I MBROJTJES DHE I SIGURISË

MASTER PROFESIONAL: SIGURI KIBERNETIKE NË FUSHËN E MBROJTJES

Zhvillimi i një IDS nëpërmjet Machine Learning

PUNUAR NGA

Rigers Zaimi

UDHËHEQËSI I TEMËS

Dr. Ulsi Meta

Viti Akademik 2024-2025

© **Copyright** Rigers Zaimi, 2025

Përmbajtja e këtij punimi është totalisht autentike. Të gjitha të drejtat e rezervuara.

Deklaratë

Unë, Rigers Zaimi deklaroj që: **(1)** Kjo mikrotezë përfaqëson punimin tim origjinal, përveç rasteve të citimeve dhe referencave dhe **(2)** Kjo mikrotezë nuk është përdorur më parë si mikrotezë apo projekt kursi në këtë Universitet apo në Universitete të tjera.

Rigers Zaimi

23.09.2025

ABSTRAKT

Siguria kibernetike përbën një sfidë madhore, pasi sulmet kibernetike po shtohen në mënyrë eksponenciale dhe po bëhen gjithnjë e më komplekse. Metodatat tradicionale të zbulimit të ndërhyrjeve, të bazuara në signatures (firma) ose rregulla fikse, shpesh kanë kufizime serioze përballë sulmeve të reja ose të sofistikuara. Kjo situatë e bën edhe më të domosdoshëm zhvillimin e sistemeve më efikase të zbulimit të ndërhyrjeve (IDS) që mund të identifikojnë dhe parandalojnë kërcënimet në kohë reale. Në këtë kuadër, kjo tezë fokusohet në ndërtimin e një IDS nëpërmjet **Machine Learning**, duke përdorur njohuri dhe pajisje të aksesueshme në domain publik nga çdo kush që ka interes.

Qëllimi kryesor është ndërtimi dhe përmirësimi gradual i një modeli IDS përmes një procesi iterativ. Fillimisht u zhvillua një model bazë i thjeshtë. Më pas, në secilin iteracion shtohen optimizime të ndryshme (si transformimi i të dhënave dhe balancimi i klasave) për të rritur saktësinë e modelit dhe për të adresuar shpërndarjen e pabalancuar të sulmeve në dataset. Ky proces kulmon me kalimin nga një model bazik në një arkitekturë me dy faza, duke arritur një përmirësim të ndjeshëm të performancës së sistemit.

Performanca e modeleve evaluohet me metrikat **F1-score**, **precision** dhe **recall**, për të siguruar një vlerësim gjithëpërfshirës të saktësisë së detektimit. Rezultatet konfirmuan efektivitetin e modelit, duke arritur nivele të kënaqshme të performancës sipas metrikave të mësipërme, duke siguruar një balancë të mirë midis precision dhe recall, si dhe duke reduktuar ndjeshëm numrin e alarmeve të rreme. Në përfundim, ndërtimi i një IDS të besueshme është një proces kompleks që kërkon përmirësim të vazhdueshëm dhe proces të kujdesshëm testimi përgjatë çdo hapi.

FALENDERIME DHE DEDIKIME

Dëshiroj të shpreh falënderimet e mia më të thella ndaj mbikëqyrësit tim, Prof. Dr. Ulsi Meta, për mbështetjen e tij të vazhdueshme, kritikën e vlefshme dhe udhëzimet profesionale gjatë gjithë procesit të hartimit të kësaj teze. Këshillat, përgatitja dhe ndihma e tij kanë qenë thelbësore në realizimin e këtij punimi praktik.

Gjithashtu, shpreh mirënjohjen time ndaj miqve të mi, Enea Xhepa dhe Marko Kushta, për ndihmën e tyre të çmuar. Ky kërkim u bë i mundur dhe falë kontributit të tyre, veçanërisht përmes sugjerimeve dhe njohurive të ndara në fushën e Machine Learning.

PËRMBAJTJA E PUNIMIT

KAPITULLI I: HYRJE.....	1
1. 1. Qëllimi i punimit.....	1
1. 2. Motivimi	1
1. 3. Objektivat.....	2
1. 4. Metodologjia.....	3
1. 5. Rishikimi i literaturës.....	4
KAPITULLI II: MACHINE LEARNING DHE SIGURIA KIBERNETIKE	5
2. 1. Avantazhet e ML	5
2. 2. Disavantazhet e ML.....	6
2. 3. Mësimi i Mbikëqyrur	6
2. 4. ML dhe lidhja me IDS.....	7
2. 5. Konfigurimi i ambientit	7
2. 5. 1. WSL dhe Përfitimet.....	7
2. 5. 2. Visual Studio Code dhe Jupyter Notebooks	10
KAPITULLI III: DATASET-I I PËRDORUR, ANALIZA E VEÇORIVE.....	13
3. 1. Importi në ambient	16
3. 2. Formatimi i mospërputhjeve	18
3. 3. Shpërndarja e Sulmeve	19
3. 4. Dataseti Train dhe Test.....	19
3. 5. Dallimet kryesore në kolonat mes dataset-it të plotë dhe dataset-eve train/test:.....	20
3. 6. Ndryshime në shpërndarje të sulmeve.....	21
3. 7. Llojet e Sulmeve.....	22
3. 8. Shpërndarja e sulmeve në 2 setet	25
3. 9. Analiza e Kolonave/Veçorive për Train/Test	26
3. 9. 1. Kontrolli i vlerave që mungojnë.....	26
3. 9. 2. Vecoritë jo-numerike	27
3. 9. 3. One-hot Encoding	29
3. 9. 4. Enkodimi i kolonave kategorike	30
3. 9. 5. Kolona Numerike	32
KAPITULLI IV: PËRGATITJA PËR KRIJIMIN E MODELIT.....	35
4. 1. Çfarë është një Pipeline.....	35
4. 2. Ndarja train/test në kontekstin e modelit.....	36
4. 2. 1. Aplikimi i .fit() dhe .transform()	36
4. 2. 2. Ndarja e seteve në komponent X dhe y	37
4. 3. Enkodimi i attack_cat	38

4. 4. Peshat e Klasave.....	39
4. 5. Metrikat e performancës.....	40
KAPITULLI V: IMPLEMENTIMI DHE TESITMI I MODELEVE.....	42
5. 1. Struktura e Modelit	42
5. 1. 1. Specifikime për ekzekutimin.....	45
5. 1. 2. Fillimi i trajnimit të modelit bazik	46
5. 1. 3. Procesi i trajnimit të modelit bazik.....	48
5. 1. 4. Interpretimi i rezultateve për modelin bazik	50
5. 2. Implementimi i OneHotEncoder	52
5. 2. 1. One Hot Encoder.....	52
5. 2. 2. Interpretimi i Modelit OHE.....	54
5. 3. Implementimi i Val Set	55
5. 3. 1. Set Val i balancuar sipas klasave	55
5. 3. 2. Reduktimit të LR dhe EarlyStopping	55
5. 3. 3. Interpretimi i Modelit me Val Set.....	56
5. 3. 4. Diferencat midis instancave për të njëjtin Model.....	59
5. 3. 5. Implementimi i Determinizmit.....	59
5. 4. Implementimi i transformimit LOG dhe Binarizimi i veçorive	60
5. 4. 1. Asimetria (Skew).....	60
5. 4. 2. Efekti i transformimit LOG në veçori	62
5. 4. 3. Trajtimi i kolonave Binare.....	65
5. 4. 4. Interpretim i rezultateve për modelin Log dhe Binar	65
5. 5. Implementimi i Modelit me 2 Faza:	68
5. 5. 1. Faza-1 Analiza Binare	70
5. 5. 2. Interpretimi i fazës 1	72
5. 5. 3. Faza-2 Analiza Kategorike	73
5. 5. 4. Interpretimi i modelit me 2 faza	75
KAPITULLI VI: KONKLUSIONE DHE REKOMANDIME.....	78
BIBLIOGRAFIA.....	82

LISTA E TABELAVE

Tabela 3-1 Veçoritë e UNSW-NB15	14
Tabela 3-2 Nurmi i vlerave NaN dhe Bosh për cdo veçori.....	27
Tabela 3-3 % e shpërndarjes së protokolleve.....	28
Tabela 5-1 Performanca e modelit bazik	50
Tabela 5-2 Performanca e modelit me OHE	54
Tabela 5-3 Krahasim mes modelit me dhe pa seeding.....	59
Tabela 5-4 Vlerat e skew për veçoritë numerike, duke zbritur	62
Tabela 5-5 Performanca e modelit para dhe pas transformimeve.....	65
Tabela 5-6 Performanca për fazën e parë të modelit.....	72
Tabela 5-7 Krahasimi i modelit Log & Bin me atë me 2 faza	75

LISTA E FIGURAVE

Figura 2-1 Lista e instalimeve WSL në host.....	8
Figura 2-2 Konfigurimi i Ubuntu në WSL	8
Figura 2-3 Output i NVIDIA-SMI.....	9
Figura 2-4 Instalimi i librarisë VENV python	9
Figura 2-5 Aktivizimi i VENV tf.....	10
Figura 2-6 Visual Studio Code.....	11
Figura 2-7 Instalimi i ipykernel	11
Figura 2-8 Pamja e EXPLORER të VS CODE	11
Figura 3-1 Skedarët e UNSW-NB15.....	13
Figura 3-2 Pie Charts për shpërndarjen e trafikut në data_full dhe df.....	22
Figura 3-3 Histogram për shpërndarjen e sulmeve	25
Figura 3-4 Distribucioni i veçorisë service.....	29
Figura 3-5 Distribucioni i veçorisë state	29
Figura 3-6 Veçoritë e enkoduara	30
Figura 5-1 Struktura e modelit bazik	44
Figura 5-2 Paraqitja e funksionit. Loss nëpër epoka.....	45
Figura 5-3 Progresi i modelit përgjatë epokave	48
Figura 5-4 Loss e modelit përgjatë epokave	49
Figura 5-5 Saktësia e modelit përgjatë epokave	49
Figura 5-6 Matrica për modelin bazik	51
Figura 5-7 Saktësia e modelit Val përgjatë epokave	56
Figura 5-8 Loss e modelit Val përgjatë epokave.....	57
Figura 5-9 Performanca e modelit Val	58
Figura 5-10 Shpërndarja Normale	60
Figura 5-11 Paraqitja e vlerave të ndryshme të skew	61
Figura 5-12 Veçoritë ct_* para transformimit LOG	63
Figura 5-13 Veçoritë ct_* pas transformimit LOG	64
Figura 5-14 Matrica e modelit pas Transformimeve.....	68
Figura 5-15 Faza 1 e modelit	70
Figura 5-16 Funkzioni Sigmoid	71
Figura 5-17 Trajnimi i modelit gjatë fazës 1	71
Figura 5-18 Matrica për fazën 1	73
Figura 5-19 Faza 2 e modelit	74
Figura 5-20 Trajnimi i fazës 2 të modelit.....	74
Figura 5-21 Matrica për modelin me 2 faza.....	77

LISTA E SHKURTIMEVE

tf	tensorflow
ML	Machine Learning
GPU	Graphics Processing Unit
WSL	Windows Subsystem for Linux
CLI	Command Line Interface
VENV	Virtual Enviornments
VS Code	Visual Studio Code
IDE	Integrated Development Environment
df	DataFrame
FP	False Positive
FN	False Negative
TP	True Positive
TN	True Negative
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
ReLU	Rectified Linear Unit
LR	Learning Rate

KAPITULLI I

HYRJJE

Siguria kibernetike është një nga sfidat më të mëdha të epokës dixhitale. Me rritjen eksponenciale të përdorimit të internetit, pajisjeve inteligjente dhe shërbimeve online, sulmet kibernetike janë bërë gjithnjë e më të shpeshta dhe më komplekse. Metodat tradicionale të zbulimit të sulmeve kibernetike, që mbështeten kryesisht në **firma (signatures)** dhe rregulla të paracaktuara, kanë treguar kufizime të mëdha përballë sulmeve të reja, të panjohura ose të sofistikuara (Sommer & Paxson, 2010).

Në këtë kontekst, **Machine Learning (ML)** është shfaqur si një zgjidhje premtuese për të përmirësuar IDS (Intrusion Detection System).

1. 1. Qëllimi i punimit

Qëllimi kryesor i kësaj pune është **zhvillimi i një IDS** nëpërmjet **Machine Learning**, që mund të zhvillohet duke përdorur **hardware dhe software të aksesueshme nga çdo individ**. Të krijohet një zgjidhje e thjeshtë, e qëndrueshme dhe me kosto të ulët, që mund të zgjerohet përtej sferës akademike nëpërmjet zhvillimit të vazhdueshëm.

1. 2. Motivimi

Teknologjitë tradicionale të menaxhimit dhe zbulimit të sulmeve kibernetike, si sistemet klasike të IDS, shpesh janë të kufizuara në aftësinë e tyre për të zbuluar sulme të reja dhe të panjohura. Në këtë kontekst, përdorimi i teknikave të ML ofron një mundësi premtuese për të përmirësuar ndjeshëm performancën e sistemeve të zbulimit të ndërhyrjeve. Modelet e ML kanë aftësinë të analizojnë sasi të mëdha të dhënash dhe të identifikojnë modele të sjelljeve anormale që mund të tregojnë për një sulm në zhvillim. Kërkesa për specialistë në këtë fushë është në rritje të vazhdueshme, dhe tregu i punës ofron një sërë mundësish për studiues dhe

profesionistë të sigurisë kibernetike që zotërojnë njohuri mbi ML dhe analizë të dhënash. E ardhmja e kësaj fushe pritet të jetë edhe më e lidhur me inteligjencën artificiale, ku sistemet e automatizuara do të luajnë një rol kyç në mbrojtjen e infrastrukturave kritike.

Zgjedhja e kësaj teme lidhet ngushtë me ndërthurjen e interesave të mia akademike dhe profesionale në fushat e **Machine Learning** dhe **Sigurisë Kibernetike**. Përvojat e mëparshme në ndërtimin e modeleve neurale, kryesisht në fushat e analizave statistikore dhe imazherisë mjekësore, kanë zhvilluar aftësitë e mia në lidhje me modelim, përpunim e të dhënave dhe interpretim të rezultateve. Këto aftësi përbëjnë një bazë solide që më ka mundësuar të lidh me sukses njohuritë e fituara në vitin e fundit mbi sigurinë kibernetike, duke krijuar një lidhje midis dy fushave. Në këtë kontekst, zhvillimi i një **IDS (Intrusion Detection System)** nëpërmjet teknikave të Machine Learning përfaqëson ndërthurjen që më lejon të aplikoj kompetencat e fituara nga programi master, si dhe të zgjeroj horizontin tim profesional nëpërmjet zhvillimit të një sistemi praktik për mbrojtje nga kërcënimet kibernetike.

1. 3. Objektivat

- Zhvillimi i një IDS të bazuar në teknika të Machine Learning, i aftë të dallojë trafikun normal nga trafiku i dyshimtë dhe të klasifikojë llojet kryesore të sulmeve kibernetike.
- Zgjedhja dhe përdorimi i një dataset-i të përshtatshëm, konkretisht UNSW-NB15, duke analizuar cilësinë e të dhënave dhe duke kryer pastrimin, transformimin dhe ndarjen e tyre për trajnimin dhe testimin e modeleve.
- Optimizimi i mëtejshëm i modelit, nëpërmjet trajtimit të disbalancës së klasave, eksperimentimit me teknika të ndryshme të përpunimit të të dhënave, dhe vlerësimit të modeleve me metrika të avancuara për të rritur performancën e sistemit.

1. 4. Metodologjia

Metodologjia e kësaj teze bazohet në testimin dhe implementimin e teknikave të optimizimit duke dhënë rezultate inkrementale:

Përzgjedhja e dataset-it: Përdoret UNSW-NB15, një dataset për IDS, i cili reflekton kushtet reale të trafikut në rrjet, klasat e sulmit të pabalancuara, duke qenë karakteristika kryesore. Është përdorur UNSW-NB15, sepse reflekton kushte reale të trafikut në rrjet, duke përfshirë klasat e sulmit të pabalancuara si karakteristikë kryesore. Përfshin një ndarje efektive train/test, e paraprosesuar nga publikuesi (Moustafa & Slay, 2015).

Përpunimi i të dhënave: Kryhet pastrim i dataset-it, transformimi i variablave (p.sh. log-transform për kolonat me shpërndarje të shtrembëruar), si dhe enkodimi i variablave kategorike përmes **One-Hot Encoding**, binarizimit etj. Implementimi i realizuar nëpërmjet pipelines për të automatizuar procesin.

Trajnimi i modelit: Zbatohet një model i bazuar në **supervised learning**, duke përdorur algoritme të avancuara klasifikimi si **rrjetet neurale**. Veçori të reja implementohen në një strukturë iterative duke testuar rezultatet gjatë çdo hapi.

Vlerësimi i performancës: Performanca e modelit do të analizohet përmes metrikave standarde si **accuracy, precision, recall, F1-score**, si dhe përmes “**confusion matrix**” për të vlerësuar kapacitetin e modelit si dhe lidhjen midis klasave të sulmit.

Të gjithë tabelat dhe grafikët që nuk përmbajnë burim janë gjeneruar nga unë, duke përdorur gjuhën e programimit Python dhe libraritë Matplotlib, Seaborn dhe Pandas në kombinim me Excel, me qëllim vizualizimin dhe analizimin e të dhënave të përfshira, duke i sjellë ato në një format më të qartë dhe të lexueshëm.

1. 5. Rishikimi i literaturës

Studimet mbi përdorimin e ML në IDS kanë treguar rezultate premtuese. Buczak & Guven 2016, ofrojnë një pasqyrë të gjerë mbi metodat e ndryshme të data mining dhe ML të aplikuara në siguri, duke theksuar rolin e tyre në uljen e alarmeve false dhe rritjen e saktësisë. Shone et al. 2018 propozuan përdorimin e **deep learning** për detektimin e trafikut keqdashës, duke demonstruar se rrjetet e thella neuronale arrijnë performancë më të lartë se metodat tradicionale.

Nga ana tjetër, Sommer & Paxson (2010) theksojnë kufizimet e përdorimit të ML në IDS, duke përmendur varësinë nga dataset-et e etiketuara dhe vështirësitë në detektimin e sulmeve të panjohura. Këto sfida janë edhe më të dukshme në dataset-e si UNSW-NB15, ku ekziston një shpërndarje e pabalancuar e klasave (Moustafa & Slay, 2015).

Megjithatë, studimet kanë sugjeruar se përdorimi i teknikave si **class_weighting**, **oversampling** dhe **feature selection** mund të përmirësojnë ndjeshëm performancën e modeleve ML për detektimin e sulmeve të vogla, por kritike (Biggio & Roli, 2018).

KAPITULLI II

MACHINE LEARNING DHE SIGURIA KIBERNETIKE

Machine Learning (ML) është një nëngrup i Inteligjencës Artificiale (AI), shfaqur si një zgjidhje premtuese për zbulimin e kërcënimeve dhe parandalimin e tyre në kohë reale (Buczak & Guven, 2016). Ndryshe nga metodat tradicionale të bazuara në instruksione ose rregulla fikse, modelet e ML trajnohen mbi dataset-e të gjera duke mësuar strukturën dhe lidhjet që ekzistojnë brenda të dhënave, sjelljet tipike të trafikut normal dhe anomalitë që lidhen me sulme të mundshme. Këto modele përdorin rrjete neurale dhe algoritme statistikore për të analizuar sasi të mëdha të dhënash, duke i bërë të aftë të identifikojnë edhe struktura që më parë ishin të panjohura si zero-day attacks. Një nga përparësitë kryesore të ML është aftësia për të përmirësuar performancën me kalimin e kohës duke e bërë procesin e detektimit dhe reagimit më të shpejtë dhe më efikas (Sommer & Paxson, 2010).

2. 1. Avantazhet e ML

Përdorimi i ML në fushën e sigurisë kibernetike sjell një sërë avantazhesh. Së pari, modelet ML arrijnë një nivel më të lartë saktësie, pasi mund të përpunojnë sasi masive të të dhënave dhe të identifikojnë karakteristikat dalluese të sulmeve, duke reduktuar numrin e alarmeve false (FP) (Shone et al., 2018). Së dyti, këto modele mundësojnë detektim në kohë reale, pasi algoritmet analizojnë rrjedhën e të dhënave në mënyrë të vazhdueshme, duke lejuar identifikimin dhe reagimin e menjëhershëm kundrejt kërcënimeve. Një tjetër avantazh i rëndësishëm është automatizimi i proceseve, që u mundëson analistëve të fokusohen në incidente më komplekse. Së fundmi, modelet ML janë të shkallëzueshme, duke u përshtatur me volume të mëdha të të dhënave dhe duke u aplikuar lehtësisht në rrjete të mëdha dhe infrastruktura kritike.

2. 2. Disavantazhet e ML

Megjithatë, përdorimi i ML has dhe vështirësi. Fillimisht, modelet janë shumë të varura nga të dhënat, duke kërkuar dataset-e të mëdha dhe të balancuara për trajnime efektive, kjo e fundit e pamundur në analizimin e trafikut të rrjeteve duke konsideruar sa të rralla janë sulmet. Gjithashtu, kualiteti i të dhënave është me rëndësi të lartë: nëse dataset-i përmban gabime ose etiketime të pasakta, performanca e modelit bie ndjeshëm (Moustafa & Slay, 2015). Një tjetër kufizim është kompleksiteti i modeleve, sidomos rrjetet e thella neurale, të cilat kërkojnë fuqi të lartë procesimi, si data centers, dhe nuk janë gjithmonë të përshtatshme për t'u vendosur në pajisje me fuqi të kufizuar. Së fundmi, ekziston rreziku i sulmeve nga kundërshtarët, ku sulmuesit manipulojnë të dhënat ose vet strukturën vetë e modelit me qëllim uljen e besueshmërisë.(Biggio & Roli, 2018).

2. 3. Mësimi i Mbikëqyrur

Një nga metodat më të përdorura në fushën e detektimit të sulmeve kibernetike është Mësimi i Mbikëqyrur (Supervised Learning). Në këtë mënyrë, dataset-i ndahet në dy pjesë kryesore: seti i trajnimit (train) dhe seti i testimit (test). Modeli përdor të dhënat e trajnimit duke mësuar marrëdhëniet ndërmjet veçorive hyrëse dhe etiketave përkatëse. Më pas, performanca e modelit vlerësohet mbi të dhënat e testimit, të cilat nuk janë përdorur gjatë trajnimit, duke mundësuar verifikimin e aftësisë së tij për të përgjithësuar (Goodfellow, Bengio, & Courville, 2016). Kjo qasje është veçanërisht efektive për detektimin e sulmeve kibernetike, sidomos kur përdoren dataset-e të etiketuara si UNSW-NB15, ku çdo rrjedhë trafiku kategorizohet me 2 lloje etiketash, **binare (label)** që tregon a është sulm, dhe **një kategorike (attack_cat)** që shpreh llojin e sulmit. Megjithatë, një kufizim i rëndësishëm është se modelet supervised kanë performancë të kufizuar ndaj sulmeve të reja (zero-day attacks), të cilat nuk janë të përfshira në të dhënat e trajnimit (Sommer & Paxson, 2010).

2. 4. ML dhe lidhja me IDS

Machine Learning përfaqëson një paradigmë të fuqishme të analizës së të dhënave, duke ofruar mekanizma të avancuara për klasifikim, parashikim dhe zbulim anomalish. Këto aftësi e bëjnë ML veçanërisht të përshtatshëm për fushën e sigurisë kibernetike, ku nevoja për identifikimin e shpejtë dhe të saktë të kërcënimeve është kritike. Në veçanti, ML është përdorur gjerësisht në ndërtimin e sistemeve IDS, të cilat kanë për qëllim monitorimin e trafikut të rrjetit dhe dallimin midis sjelljes normale dhe atyre potencialisht keqdashëse. Ky aplikim do të analizohet në detaje në kapitujt në vijim, ku fokusi do të jetë në përdorimin e metodave supervised mbi dataset-in UNSW-NB15.

2. 5. Konfigurimi i ambientit

Tema do të realizohet në një laptop Acer Helios Predator 300, Nr. Modeli: PH315-54. Ky laptop ka një GPU të dedikuar, që do të përdoret për trajnimin e modelit. Specifikimet

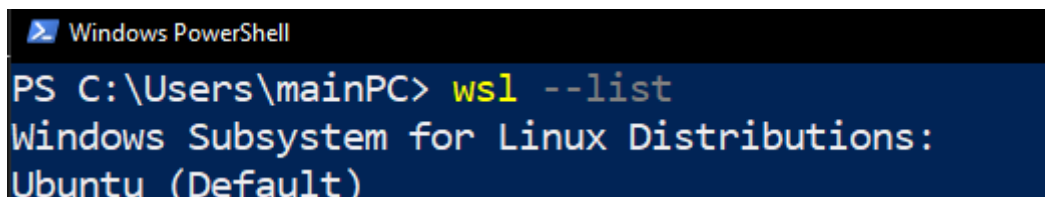
Hardware të pajisjes HOST:

- GPU: NVIDIA RTX 3070 Mobile 8GB VRAM
- CPU: INTEL I7-11800H
- RAM: 16 GB DDR4

2. 5. 1. WSL dhe Përfitimet

Zhvillimi i modelit do të realizohet me anë të përdorimit të librisë **tensorflow** për **python**.

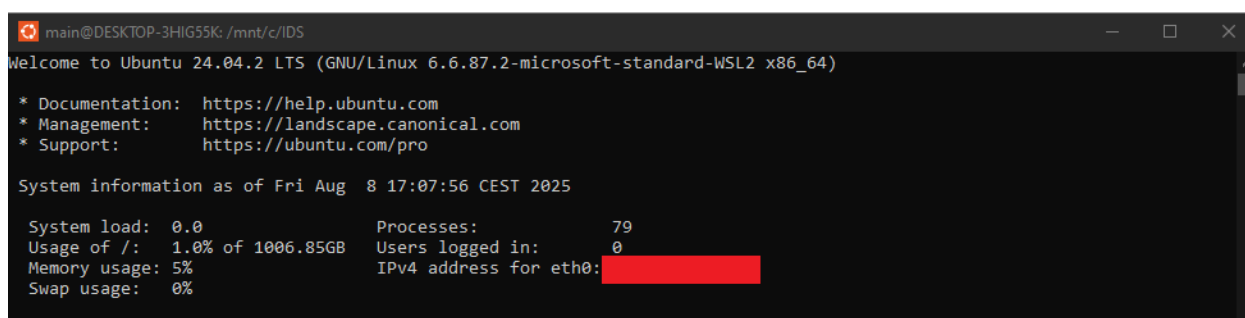
Çdo **version > 2.11** i **tensorflow** nuk suporton më përdorimin e **GPU** në mënyrë native në Windows. Do të përdor WSL (Windows Subsystem for Linux) për të virtualizuar një ambient Ubuntu ku do të konfigurojë ambientin ndërkoh që sistemi “host” është akoma Windows.



```
Windows PowerShell
PS C:\Users\mainPC> wsl --list
Windows Subsystem for Linux Distributions:
Ubuntu (Default)
```

Figura 2-1 Lista e instalimeve WSL në host

Kam instaluar versionin e fundit të Ubuntu. WSL konfiguron vetëm sisteme bazike me virtualizim prandaj ekziston vetëm një ndërfaqe CLI për te vepruar me Ubuntu.



```
main@DESKTOP-3HIG55K: /mnt/c/IDS
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Aug 8 17:07:56 CEST 2025

System load:  0.0      Processes:      79
Usage of /:   1.0% of 1006.85GB   Users logged in: 0
Memory usage: 5%      IPv4 address for eth0: 
Swap usage:  0%
```

Figura 2-2 Konfigurimi i Ubuntu në WSL

Trajnimi i të dhënave do të realizohet me GPU. Në ML, shumica e llogaritjeve janë **matricore/tensor** (p.sh. shumëzime matricash), të cilat mund të ndahen në shumë operacione të vogla dhe të kryhen paralelisht. GPU përbehen nga mijëra bërthama të vogla të specializuara në kryerjen e llogarive të thjeshta me shpejtësi të lartë. RTX 3070 mobile, e cila do përdoret për të trajnuar modelet, ka **5120 Cuda Cores**. Për këtë arsye GPU ka fuqi procesimi më të madhe.

Komanda **nvidia-smi** nxjerr informacion në lidhje me kartat grafike prezentë në kompjuter. Në këtë rast vëzhgohet që Ubuntu nëpërmjet WSL e identifikon GPU si RTX 3070 Mobile. Në total ka 8GB VRAM, një faktor i rëndësishëm gjatë procesit të trajnimit sepse dikton madhësinë e modelit dhe shpejtësinë e procesit të trajnimit.

```
(tf) main@DESKTOP-3HIG55K:/mnt/c/IDS$ nvidia-smi
Fri Aug 8 17:54:13 2025
```

NVIDIA-SMI 580.65.05			Driver Version: 580.88			CUDA Version: 13.0		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M. MIG M.	
0	NVIDIA GeForce RTX 3070	On	00000000:01:00.0	Off	0%	Default	N/A	
N/A	53C	P8	13W / 110W	144MiB / 8192MiB			N/A	

```

Processes:
  GPU  GI  CI          PID  Type  Process name                        GPU Memory
   ID   ID               Type                               Usage
-----
No running processes found

```

Figura 2-3 Output i NVIDIA-SMI

```
(tf) main@DESKTOP-3HIG55K:/mnt/c/IDS$ nvidia-smi --query-gpu=name,memory.total,memory.free,memory.used,driver_version,compute_cap --format=csv
name,memory.total [MiB], memory.free [MiB], memory.used [MiB], driver_version, compute_cap
NVIDIA GeForce RTX 3070 Laptop GPU, 8192 MiB, 7603 MiB, 418 MiB, 580.88, 8.6
```

Ubuntu vjen me **python3** të instaluar. Për menaxhim më efektiv të ambientin mund të përdoret libraria **VENV**, e cila lejon krijimin e instancave të shumëfishta **python**. Kjo lejon testimin e konfigurimeve të ndryshme, dhe gjithashtu paketimin e një ambienti ekzistues për distribucion të programit pa patur nevojën që një përdorues të instaloi python në pajisjen e tyre. Zhvillimi i mëtejshëm i kësaj teme do të realizohet duke përdorur venv, procesi i inicializimit shpjegohet më poshtë.

Figura 2-4 Instalimi i librisë VENV python

```
main@DESKTOP-3HIG55K: ~$ sudo apt install python3.12-venv
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-pip-whl python3-setuptools-whl
The following NEW packages will be installed:
  python3-pip-whl python3-setuptools-whl python3.12-venv
0 upgraded, 3 newly installed, 0 to remove and 13 not upgraded.
Need to get 2425 kB of archives.
After this operation, 2771 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-pip-whl all 24.0+dfsg-1ubuntu1.2 [1703 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-setuptools-whl all 68.1.2-2ubuntu1.2 [716 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3.12-venv amd64 3.12.3-1ubuntu0.7 [5680 B]
Fetched 2425 kB in 7s (370 kB/s)
Selecting previously unselected package python3-pip-whl.
(Reading database ... 40787 files and directories currently installed.)
Preparing to unpack .../python3-pip-whl_24.0+dfsg-1ubuntu1.2_all.deb ...
Unpacking python3-pip-whl (24.0+dfsg-1ubuntu1.2) ...
Selecting previously unselected package python3-setuptools-whl.
Preparing to unpack .../python3-setuptools-whl_68.1.2-2ubuntu1.2_all.deb ...
Unpacking python3-setuptools-whl (68.1.2-2ubuntu1.2) ...
Selecting previously unselected package python3.12-venv.
Preparing to unpack .../python3.12-venv_3.12.3-1ubuntu0.7_amd64.deb ...
Unpacking python3.12-venv (3.12.3-1ubuntu0.7) ...
Setting up python3-setuptools-whl (68.1.2-2ubuntu1.2) ...
Setting up python3-pip-whl (24.0+dfsg-1ubuntu1.2) ...
Setting up python3.12-venv (3.12.3-1ubuntu0.7) ...
```

Pas instalimit të paketës krijohet një ambient i ri (virtual environment) të ri me emrin **tf**, shkurt për tensorflow.



```
main@DESKTOP-3HIG55K: ~  
main@DESKTOP-3HIG55K:~$ python3 -m venv tf  
main@DESKTOP-3HIG55K:~$ source tf/bin/activate  
(tf) main@DESKTOP-3HIG55K:~$
```

Figura 2-5 Aktivizimi i VENV tf

Komanda ***source tf/bin/activate*** navigon te skedari ku ndodhet **ambienti virtual** dhe e aktivizon atë. Kjo dallohet se tani në CLI tregohet **(tf)**. Nativisht, tensorflow nuk vepron me GPU, prandaj duhet të instalohet libraria CUDA nga NVIDIA. Drivers për GPU të familjes RTX vijnë me libaritë të cilat mundësojnë përdorimin e CUDA Cores nga **tf**.

Instalohet paketa **tensorflow[and-cuda]**, nëpërmjet **pip**. Kjo paketë lejon tensorflow të përdor GPU.

2. 5. 2. Visual Studio Code dhe Jupyter Notebooks

VS Code është IDE që do përdoret për procesin e programimit, zhvillimin e modelit, procesimin e të dhënave, krijimit të tabelave, grafikëve dhe ekzekutimin.

Është një program falas nga Microsoft me module që suportojnë **python**. Në vend të formatit standart **.py** për python, do të përdoret **Jupyter Notebooks**. Është një mjet i fuqishëm që lejon ekzekutimin e kodit nëpërmjet blloqeve. Suporton forma të ndryshme output-i si tekst, html, markdown të cilat janë të nevojshme për vizualizimin e grafikëve.

Projekti duhet të vendoset brenda sistemit virtual dhe jo atij të Windows. Në të kundërt do të ketë probleme performance ku vetë WSL bëhet “bottleneck” gjatë transferimit të të dhënave midis 2 sistemeve. Për këtë projekt janë vendosur brenda skedarit **~/IDS**.

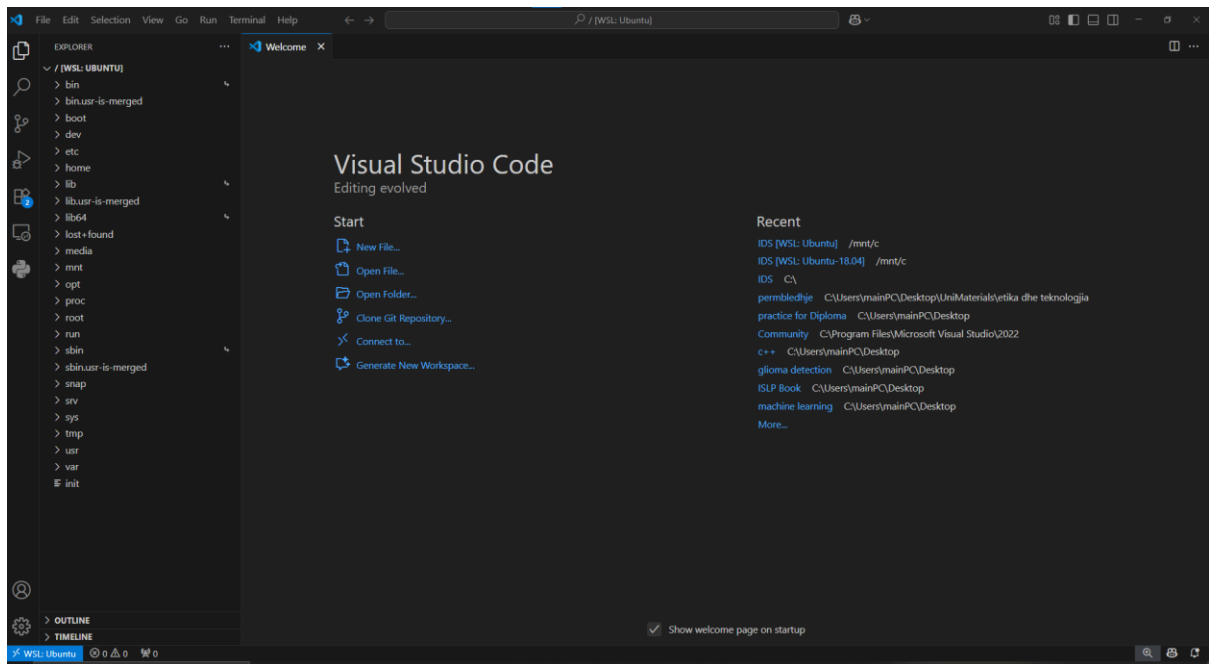


Figura 2-6 Visual Studio Code

Duhet regjistruar **venv tf** si një python kernel për tu njohur nga **VS Code**.

```
main@DESKTOP-3HIG55K: /mnt/c/IDS
(tf) main@DESKTOP-3HIG55K:/mnt/c/IDS$ python -m ipykernel install --user --name=tf --display-name "Python (tf)"
Installed kernelspec tf in /home/main/.local/share/jupyter/kernels/tf
(tf) main@DESKTOP-3HIG55K:/mnt/c/IDS$
```

Figura 2-7 Instalimi i ipykernel

Me komandën **code .** hapet **VS Code** nëpërmjet WSL.

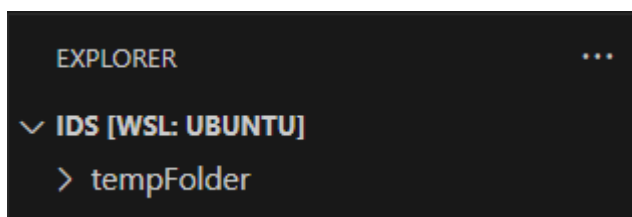


Figura 2-8 Pamja e EXPLORER të VS CODE

GPU_Test.ipynb kontrollon n.q.s. tensorflow njeh GPU e sistemit. Kodi i ekzekutuar poshtë kthen një listë me GPU që njihen nga tensorflow.

```
import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Tensorflow njih me sukses GPU e vetëm prezent në sistem. Ambienti është gati për fillimin e punës.

KAPITULLI III

DATASET-I I PËRDORUR, ANALIZA E VEÇORIVE

Dataset-i **UNSW-NB15** është një koleksion i madh të dhënash të trafikut të rrjetit, i gjeneruar në laboratorin **Cyber Range** të **Australian Centre for Cyber Security (ACCS)**. Trafiku u gjenerua duke përdorur mjetin **IXIA PerfectStorm** për të krijuar një përzierje trafiku normal (të sigurt) dhe trafikut të dëmshëm (disa kategori të ndryshme sulmesh). Paketat e kapura (PCAP) përpunohen përmes **tcpdump** dhe më pas nxirren 49 veçori (features) që përshkruajnë aspektet e trafikut si protokollet, koha e ekzekutimit, a u përdor https, statistika për lidhjet, etj. Këto të dhëna janë përmbledhur në 4 skedarë **UNSW-NB15_X**.

Dataset-i përmban rreth 100 GB të dhëna origjinale, të ndara në disa skedarë CSV, ku secili rresht paraqet një rrjedhë (flow) të rrjetit. ***Sulmet janë të kategorizuara në 9 kategori (attack_cat)***, të cilat do të analizohen në një seksion të mëposhtëm. Ky organizim e bën dataset-in të pasur dhe gjithëpërfshirës duke siguruar varietet të lartë për zhvillimin e një IDS efektive.

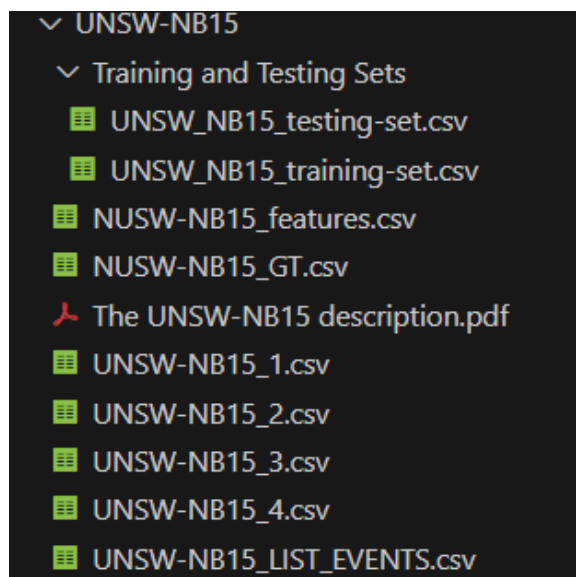


Figura 3-1 Skedarët e UNSW-NB15

Tabela 3-1 Veçoritë e UNSW-NB15

Nr.	Veçoria	Përshkrim
1	srcip	Adresa IP e burimit
2	sport	Numri i portës së burimit
3	dstip	Adresa IP e destinacionit
4	dsport	Numri i portës së destinacionit
5	proto	Protokolli i transaksionit
6	state	Tregon gjendjen dhe protokollin përkatës; p.sh. ACC, CLO, CON, ECO, ECR, FIN, INT, MAS, PAR, REQ, RST, TST, TXD, URH, URN, dhe (-) nëse 'state' nuk përdoret
7	dur	Kohëzgjatja totale komunikimit mes burimit dhe destinacionit
8	sbytes	Bytes të transaksionit nga burimi në destinacion
9	dbytes	Bytes të transaksionit nga destinacioni në burim
10	sttl	Vlera TTL nga burimi në destinacion
11	dttl	Vlera TTL nga destinacioni në burim
12	sloss	Paketat e burimit të ri-dërguara ose të humbura
13	dloss	Paketat e destinacionit të ri-dërguara ose të humbura
14	service	Shërbimi (p.sh. http, ftp, smtp, ssh, dns, ftp-data, irc) dhe (-) nëse nuk përdoret/dihet
15	Sload	Bit/s nga burimi
16	Dload	Bit/s nga destinacioni
17	Spkts	Numri i paketave nga burimi në destinacion
18	Dpkts	Numri i paketave nga destinacioni në burim
19	swin	Vlera e dritares TCP të burimit
20	dwin	Vlera e dritares TCP të destinacionit
21	stcpb	Numri bazë i sekuencës TCP të burimit
22	dtcpb	Numri bazë i sekuencës TCP të destinacionit
23	smeansz	Madhësia mesatare e paketave të transmetuara nga burimi
24	dmeansz	Madhësia mesatare e paketave të transmetuara nga destinacioni
25	trans_depth	Thellësia e 'pipeline' në lidhjen e request/response HTTP
26	res_bdy_len	Madhësia e pakompresuar e body së dërguar nga shërbimi HTTP i serverit

27	Sjit	Jitter i burimit (ms)
28	Djit	Jitter i destinacionit (ms)
29	Stime	Koha e fillimit të regjistrimit
30	Ltime	Koha e fundit e regjistrimit
31	Sintpkt	Koha ndërmjetëse mes paketave nga burimi (ms)
32	Dintpkt	Koha ndërmjetëse mes paketave nga destinacioni (ms)
33	tcprrt	Koha vajtje-ardhje për vendosjen e lidhjes TCP; synack + ackdat
34	synack	Koha ndërmjet paketave SYN dhe SYN_ACK gjatë vendosjes së lidhjes TCP
35	ackdat	Koha ndërmjet SYN_ACK dhe ACK gjatë vendosjes së lidhjes TCP
36	is_sm_ips_ports	Nëse adresa IP e burimit dhe destinacionit si edhe portat respektive janë të barabarta → 1, përndryshe 0
37	ct_state_ttl	Nr. për çdo state sipas intervaleve të veçanta të vlerave TTL të burimit/destinacionit
38	ct_flw_http_mthd	Nr. i rasteve që përdorin metodat GET dhe POST në service HTTP
39	is_ftp_login	Nëse sesioni FTP aksesohet me user dhe password → 1; ndryshe 0
40	ct_ftp_cmd	Nr. i rasteve që kanë CMD hapur në sesionin FTP
41	ct_srv_src	Numri i lidhjeve me të njëjtin shërbim (14) dhe adresë burimi (1) brenda 100 lidhjeve sipas kohës së fundit (26)
42	ct_srv_dst	Numri i lidhjeve me të njëjtin shërbim (14) dhe adresë destinacioni (3) brenda 100 lidhjeve sipas kohës së fundit (26)
43	ct_dst_ltm	Numri i lidhjeve të së njëjtës adresë destinacioni (3) në 100 lidhje sipas kohës së fundit (26)
44	ct_src_ltm	Numri i lidhjeve të së njëjtës adresë burimi (1) në 100 lidhje sipas kohës së fundit (26)
45	ct_src_dport_ltm	Numri i lidhjeve me të njëjtën adresë burimi (1) dhe portë destinacioni (4) në 100 lidhje sipas kohës së fundit (26)
46	ct_dst_sport_ltm	Numri i lidhjeve me të njëjtën adresë destinacioni (3) dhe portë burimi (2) në 100 lidhje sipas kohës së fundit (26)
47	ct_dst_src_ltm	Numri i lidhjeve me të njëjtin burim (1) dhe të njëjtën adresë destinacioni (3) në 100 lidhje sipas kohës së fundit (26)
48	attack_cat	Emri i kategorisë së sulmit; në këtë dataset ka 9 kategori p.sh. Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode dhe Worms
49	Label	0 për regjistrime normale dhe 1 për regjistrime të sulmeve

Burimi: (Moustafa & Slay, 2015)

3. 1. Importi në ambient

Importimi në python realizohet me anë të librarisë **pandas** e cila lejon leximin e të dhënave nga lloje të shumta skedarësh (në rastin e UNSW-NB15 .csv, .xlsx).

Automatikisht, të dhënat e lexuara ruhen në objekte të tipit **DataFrame**, të cilat janë struktura tabelare (organizuar me rreshta dhe kolona), të ndërtuara mbi **NumPy arrays**. Çdo kolonë në DataFrame ka një tip të dhënash të caktuara (numerik, tekstual, kategorik etj.), ndërsa rreshtat indeksohen për të mundësuar akses të shpejtë dhe manipulim efikas. Mbi këto struktura mund të aplikohen funksione të integruara ose të personalizuara për lexim, pastrim, transformim, grupim dhe analizë të të dhënave. Importohet 'UNSW-NB15_1.csv' nëpërmjet funksionit **pd.read_csv()**. Ruhet në objektin `data1`, i shfaqur më poshtë:

```
data1 = pd.read_csv('UNSW-NB15_1.csv')
data1.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700000 entries, 0 to 699999
Data columns (total 49 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   59.166.0.0            700000 non-null object  
1   1390                  700000 non-null object  
2   149.171.126.6        700000 non-null object  
3   53                    700000 non-null object  
4   udp                   700000 non-null object  
5   CON                   700000 non-null object  
6   0.001055              700000 non-null float64  
7   132                   700000 non-null int64  
8   164                   700000 non-null int64  
9   31                    700000 non-null int64  
10  29                    700000 non-null int64  
11  0                     700000 non-null int64  
12  0.1                   700000 non-null int64  
13  dns                   700000 non-null object
```

Output i funksionit **data1.info()** shfaqet si problematik. Kolona 0 shpreh një adresë IP, 1 tregon një numër dhe rreshti 2 përsëri shfaq një adresë IP. Skedarët e UNSW nuk përmbajnë emrat e

kolonave, është e nevojshme importimi nga **NUSW-NB15_features.csv**. Kjo realizohet në kodin e mëposhtëm.

```
features = pd.read_csv('NUSW-NB15_features.csv', encoding = 'latin1')
feature_list = features.Name.to_list()
print(f'Gjatesia e \'feature list\': {len(feature_list)} ')
print(feature_list)

Gjatesia e 'feature list': 49
['srcip', 'sport', 'dstip', 'dsport', 'proto', 'state', 'dur', 'sbytes', 'dbytes',
'sttl', 'dttl', 'sloss', 'dloss', 'service', 'Sload', 'Dload', 'Spkts', 'Dpkts',
'swin', 'dwin', 'stcpb', 'dcpb', 'smeansz', 'dmeansz', 'trans_depth',
'res_bdy_len', 'Sjit', 'Djit', 'Stime', 'Ltime', 'Sintpkt', 'Dintpkt', 'tcprrt',
'synack', 'ackdat', 'is_sm_ips_ports', 'ct_state_ttl', 'ct_flw_http_mthd',
'is_ftp_login', 'ct_ftp_cmd', 'ct_srv_src', 'ct_srv_dst', 'ct_dst_ltm', 'ct_src_
ltm', 'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'attack_cat',
'Label']
```

Për të kryer analizën e plotë bashkohen të 4 skedarët nëpërmjet **pd.concat()**. Merr si argumente listën me dataframe-t. Bashkimi kryhet sipas kolonave.

data_full përbehet nga **2,540,047 rreshta (indeksi, trafiku i monitoruara)** dhe **49 kolona (veçoritë që u listuan më sipër, tabela 3-1)**. Madhësia e **dataset-it** është **969.0 MB**. Veçori me rëndësi të lartë janë **attack_cat** dhe **Label** që shprehin llojin e sulmit. **Label** është një veçori binare, 0 → Trafiku është Normal, 1 → Trafiku është pjesë e një sulmi. **attack_cat** është një veçori kategorike, tregon se cili është lloji i sulmit të monitoruar.

```
data_full = pd.concat([data1, data2, data3, data4], ignore_index=True)
data_full.info()

<class 'pandas.core.frame.DataFrame'>
Index: 2540047 entries, 0 to 440043
dtypes: float64(12), int64(28), object(9)
memory usage: 969.0+ MB
```

.unique() sjell të gjitha vlerat unike në dataset. Vlera e parë që haset është **NaN** (Not a Number). Kryesisht përdoret për të shprehur të dhëna që mungojnë ose që janë të papërcaktuara, por në datasetin UNSW-NB15, **NaN** përdoret për të treguar që kemi aktivitet normal. Para procesimit është mirë që kjo vlerë të zëvendësohet me të njëjtin datatype si të

tjerat në kolonë, **object string**. 87.35% e dataset-it përbëhet nga aktivitet trafiku normal që nuk është i dëmshëm.

3. 2. Formatimi i mospërputhjeve

```
data_full['attack_cat'].unique()
array([NaN, 'Exploits', 'Reconnaissance', 'DoS', 'Generic', 'Shellcode',
       ' Fuzzers', 'Worms', 'Backdoors', 'Analysis', ' Reconnaissance ',
       'Backdoor', ' Fuzzers ', ' Shellcode '], dtype=object)
```

Duhet pasur kujdes nga formatimi jo konsistent. Disa prej hyrjeve në kolonën *attack_cat* kanë hapësira boshe ose ndryshime në strukturë. Lista e ndryshimeve:

- 'Reconnaissance' dhe ' Reconnaissance ' – Hapësira boshe ndërmjet ‘’.
- 'Backdoor' dhe 'Backdoors' – Shtimi i ‘s’.
- 'Shellcode' dhe ' Shellcode ' – Hapësira boshe ndërmjet ‘’.
- ' Fuzzers' dhe ' Fuzzers ' – Një hapësirë boshe në fillim.

Këto mospërputhje duhet të rregullohen para analizës.

```
data_full['attack_cat'] = (
    data_full['attack_cat']
    .astype(str)
    .str.strip()
    .str.capitalize()
    .replace({
        'Backdoors': 'Backdoor',
        'Dos': 'DoS'
    })
)
```

Gabimet e formatimit u rregulluan:

```
data_full['attack_cat'].unique()
array(['Normal', 'Exploits', 'Reconnaissance', 'Dos', 'Generic',
       'Shellcode', 'Fuzzers', 'Worms', 'Backdoor', 'Analysis'],
      dtype=object)
```

3.3. Shpërndarja e Sulmeve

Më poshtë tregohet shpërndarja e kategorive të trafikut bazuar në veçorinë *attack_cat*.

87.35% e trafikut është *Normal* dhe pjesa tjetër i përket kategorive të ndryshme sulmesh.

```
# Llogarit numrin e sulmeve/rreshtave
nr_rreshta = len(data_full)
# Shton kolonën për % totale
frek_sulmi = data_full['attack_cat'].value_counts(dropna=False).to_frame('Sasia')
frek_sulmi['% total'] = frek_sulmi['Sasia'].apply(lambda x: (x / nr_rreshta) *
100)
frek_sulmi
```

	Sasia	% total
attack_cat		
Normal	2218764	87.351297
Generic	215481	8.483347
Exploits	44525	1.752920
Fuzzers	24246	0.954549
Dos	16353	0.643807
Reconnaissance	13987	0.550659
Analysis	2677	0.105392
Backdoor	2329	0.091691
Shellcode	1511	0.059487
Worms	174	0.006850

3.4. Dataseti Train dhe Test

Dataseti i UNSW-NB15 përmban 2 skedarë me data për **trajnim (UNSW-NB15-training-set.csv)** dhe **testim (UNSW-NB15-testing-set.csv)** të modelit. Këto sete janë të paraprosesuara, kanë një shpërndarje më të balancuar midis trafikut normal dhe sulmeve, përgatitur specifikisht për trajnimin e modeleve ML.

Në UNSW-NB15, dataset-i i plotë përmban kolonat për këto aspekte të trafikut në rrjet:

- Tipare të përgjithshme të trafikut (srcip, sport, dstip, dsport, proto, state, dur, etj.)
- Tipare statistikore (spkts, dpkts, sbytes, dbytes, rate, sttl, dttl, etj.)
- Tipare të bazuara në sjellje (ct_state_ttl, ct_flw_http_mthd, ct_srv_src, etj.)
- Etiketat (attack_cat – lloji i sulmit, Label – 0 për normal, 1 për sulm).

3. 5. Dallimet kryesore në kolonat mes dataset-it të plotë dhe dataset-eve

train/test:

- Përmbajtja e kolonave nuk është e njëjtë – Train dhe Test ndajnë të njëjtën strukturë kolonash me njeri-tjetrin (45 kolona), por jo me setin original (49 kolona).
- Train dhe Test përmbajnë më pak të dhëna – Përveçse diferencave në numrin e kolonave, ndryshon dhe përmbajtja. Train dhe Test rezultojnë nga selektimi dhe procesimi i setit original (shpërndarje më diverse për trafikun dhe llojet e sulmeve).

Kodi poshtë paraqet 2 setet train dhe test. Janë bashkuar në një strukturë të vetme **df**, për analizim më të thjeshtë dhe vizualizim nëpërmjet grafikëve.

```
# Importo setet për trajnim dhe testim
data_train = pd.read_csv('Training and Testing Sets/UNSW_NB15_training-set.csv')
data_test = pd.read_csv('Training and Testing Sets/UNSW_NB15_testing-set.csv')

# Bashkoi në një dataframe
df = pd.concat([data_train, data_test], ignore_index = True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 257673 entries, 0 to 82331
Data columns (total 45 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    257673 non-null int64
1   dur                   257673 non-null float64
2   proto                 257673 non-null object
3   service               257673 non-null object
4   state                 257673 non-null object
5   spkts                 257673 non-null int64
6   dpkts                 257673 non-null int64
7   sbytes                257673 non-null int64
8   dbytes                257673 non-null int64
9   rate                  257673 non-null float64
...
43  attack_cat            257673 non-null object
44  label                 257673 non-null int64
dtypes: float64(11), int64(30), object(4)
memory usage: 90.4+ MB
```

Subseti i procesuar për trajnim ka 45 kolona, ndryshe nga i ploti që përmban 49. Funkzioni poshtë paraqet kolonat që ndodhen në dataset-in e plotë por jo në **df (bashkimi i train dhe test)**. Pjesa më e madhe e kolonave që rezultojnë vijnë prej diferencave të formatimit dhe janë prezente në **data_full dhe df**.

```
kol_ndryshe = set(col.lower() for col in df.columns) - set(col.lower() for col in
data_full.columns)
print(kol_ndryshe)

{'rate', 'dmean', 'ct_src_ltm', 'sinpkt', 'response_body_len', 'id', 'dinpkt',
'smean'}
```

Ndryshime vetëm në emërtim:

- Smeansz → smean
- Dmeansz → dmean
- res_bdy_len → response_body_len
- Sjitt → sjit
- Sintpkt → sinpkt
- Dintpkt → dinpkt

Kolona ID ka shërben vetëm për indeksimin e rreshtave dhe mund të hiqet.

Dataset-i i plotë përmban 2 vecori, **stime** dhe **ltime**, të cilat mungojnë në **train/test**. Në vendin e tyre është shtuar kolona Rate, vlerat e të cilës llogariten sipas formulës: $\frac{sbytes+dbytes}{dur}$.

3. 6. Ndryshime në shpërndarje të sulmeve

Nga grafiku shihet që **df (kombinimi i setit train dhe test)** është më i balancuar, pasi përmban një përqindje shumë më të vogël të trafikut **normal (36.1%)** krahasuar me dataset-in e plotë **(87.4%)**. Kjo rishpërndarje e të dhënave rrit mundësinë e gabimeve të **tipit I / FP** pra identifikimin e gabuar të trafikut normal si sulm. Megjithatë, një rritje e gabimeve të tipit I shoqërohet me ulje të gabimeve të **tipit II / FN**, që shpreh identifikimin e trafikut të dëmshëm si normal. Kalimi i trafikut të dëmshëm në rrjet përbën një rrezik më të lartë sigurie sesa bllokimi i trafikut legjitim, i cili **mund të lejohet përsëri në rrjet pas një verifikimi manual**.

Parandalimi i trafikut keqdashës mbetet prioriteti kryesor dhe kompromisi i sipërm është i pranueshëm.

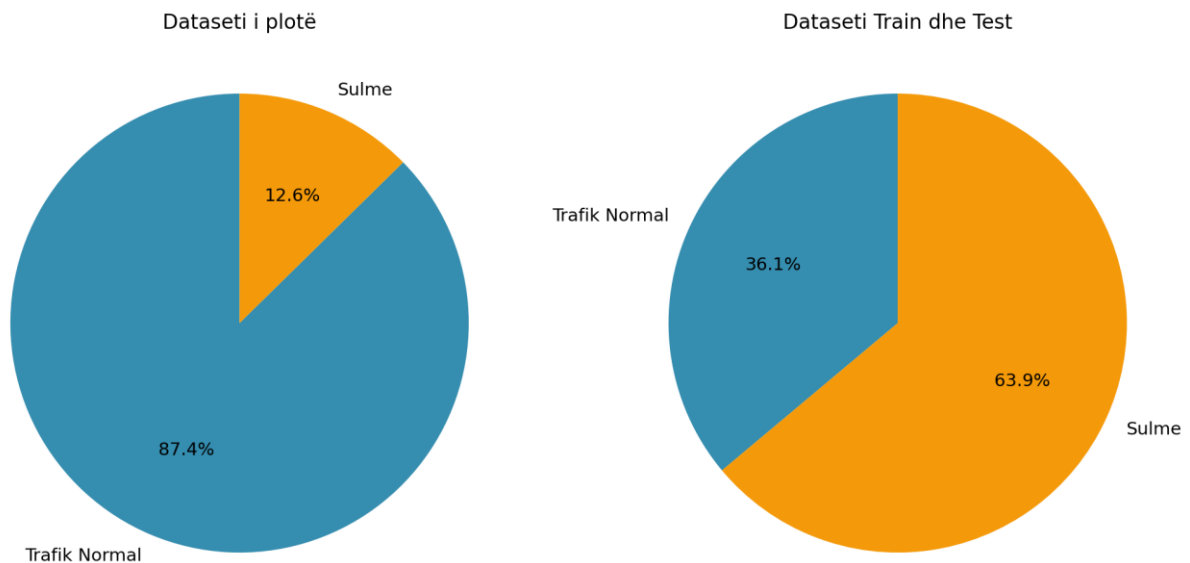


Figura 3-2 Pie Charts për shpërndarjen e trafikut në data_full dhe df

3. 7. Llojet e Sulmeve

1) Generic

Përfshin trafik të njohur keqdashës. Jo një teknikë specifike, por një përmbledhje e “signatures” të sulmeve më të njohura / të zakonshme. Në rrjet zakonisht kanë trafik shumë të lartë me diversitet të ulët. **Tregues:** Komunikim i gjatë (vecorit me ct_* kanë vlera të larta), raport asimetric $\frac{sbytes}{dbytes}$ (host dërgon më shumë data sesa merr ose kërkohet nga serveri), frekuencë e lartë e rrjedhave drejt të njëjtit host/shërbim.

2) Exploits

Shfrytëzim i dobësive të ndryshme që hasen në një rrjet, sisteme, pajisje (buffer overflow, SQLi, RCE, etj.) për të marrë akses, eskalim privilegjesh, ekzekutim kodi. Shfaqen si paketa

HTTP me payload të pazakontë (URL/query shumë i gjatë, body jo standart), ose sekuenca TCP jashtë normave (mbyllje e papritur komunikimi, ndryshim i nr. të paketave jo sipas **sequence number**). **Tregues:** Vecoria e shërbimi është HTTP/FTP/SMTP/SSH; trans_depth dhe res_bdy_len jo tipike; state anomale pas një kërkesë; rritje e ct_flw_http_mthd me metoda jo të zakonshme.

3) Fuzzers

Dërgim i lartë paketash (spam) për të bërë bruteforce ose për të forcuar një sistem të gabojë ose të bëjë crash. Shumë kërkesa të ngjashme me parametra të anormale, karaktere jo standarde dhe paketa me body jo standard. **ct_srv_dst/ct_srv_src** të larta, **dloss/sloss të shtuara (paketa "dropped")**, state ndryshon vazhdimisht.

4) DoS (Denial of Service)

Mbytja e një rrjeti/pajisje me trafik për ta bërë të papërdorshëm (SYN flood, HTTP flood, etj.) Sasi e lartë trafiku nga 1 ose burime të ndryshme. **Tregues: Spkts/Dpkts** shumë të larta, **Sload/Dload** rriten, **ct_dst_ltm** i lartë, **tcprtt/synack** të çrregullta.

5) Reconnaissance

Zbulimi dhe formimi i strukturës së një rrjeti përmes teknikave të skanimit (port scanning, zbulimi i host). Lidhje të shkurtëra kundrejt shumë portave që bëhen drop papritur. Payload-et janë të vegjël. **Tregues: ct_src_dport_ltm/ct_dst_sport_ltm** të larta, **smeansz/dmeansz** të vogla, shumë RST/REJ, **dur** shumë i shkurtër.

6) Analysis

Trafik agresiv që përdoret për skanim të një rrjeti dhe është jashtë normave normale. Kërkesa të fokusuara në një shërbim me parametra jo tipikë, p.sh. header të pazakontë, metoda të rralla.

Tregues: `ct_flw_http_mthd` me metoda jo të zakonshme, `trans_depth` anormal, `res_bdy_len` i pazakontë, kombinime të rralla **service** dhe **state**.

7) Backdoor

Linja të fshehta komunikimi në rrjet/pajisje (reverse shell etj). Kryhen lidhje të rregullta nga brenda kundrejt hosteve/serverëve të jashtëm (beaconing), nëpërmjet portava të pazakonta trafik i ulët por i qëndrueshëm. **Tregues:** `sttl/dttl` konsistente, `ct_dst_src_ltm` rritje midis dy hosteve të njëjtë, vlera të vogla për `sbytes/dbytes`.

8) Shellcode

Përpjekje për të injektuar/ekzekutuar shellcode përmes shfrytëzimit të një vulnerabiliteti. Payload përmban data binare ose hex për tu fshehur. Shoqërohet shpesh me sulme Exploit.

Tregues: `res_bdy_len/smeansz` nuk ndjekin shpërndarje normale, shfaqet bashkë me rritje të `ct_flw_http_mthd` te metoda të veçanta.

9) Worms

Lloj **malware** që përhapet në mënyrë automatike në rrjet duke skanuar dhe infektuar hoste të rinj. Skanime kundrejt një numër të madh adresave IP, në mënyrë periodike. **Tregues:** `ct_src_dport_ltm` shumë i lartë, dur i shkurtër, shumë lidhje të dështuara (p.sh. RST).

(Moustafa & Slay, 2015)

3. 8. Shpërndarja e sulmeve në 2 setet

Histogrami i mëposhtëm tregon shpërndarjen e sulmeve midis dy dataset-eve, ai i plotë dhe ndarja train/test.

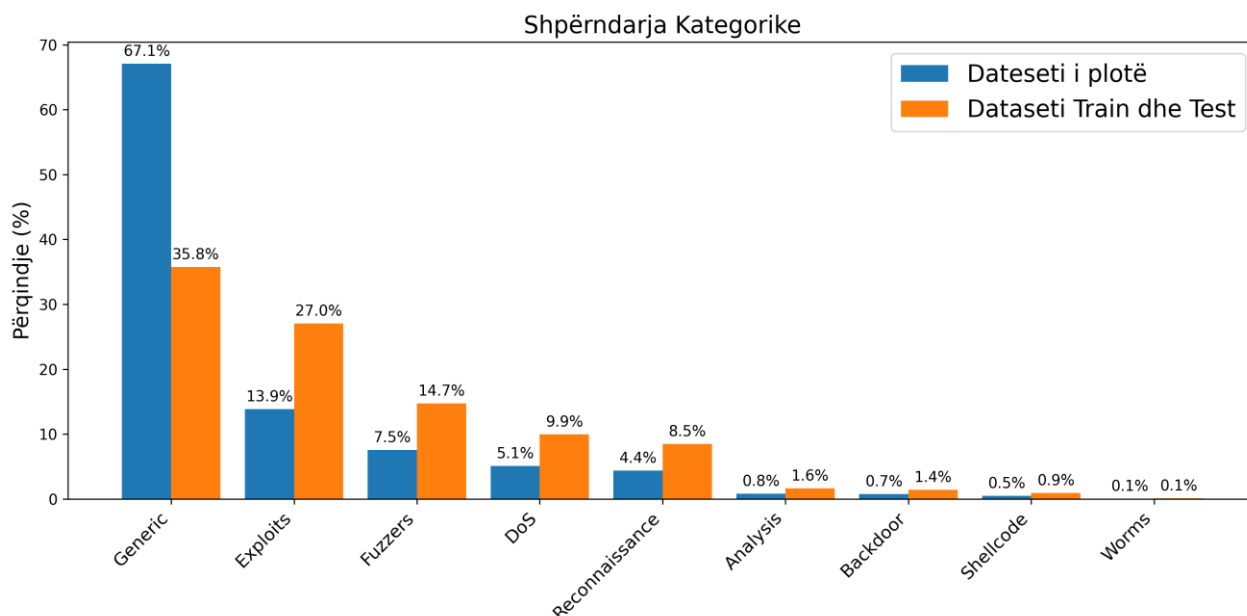


Figura 3-3 Histogram për shpërndarjen e sulmeve

1. Klasa “Generic”

Në datasetin e plotë përbën shumicën e rasteve (**67.1%**). Në setin e trajnimit/testimit kjo përqindje bie në **35.8%**, duke reduktuar dominimin e kësaj klase. Ky ndryshim rrit balancën e sulmeve që të ndihmoi modelin në mësim.

2. Klasat “Exploits” dhe “Fuzzers”

Kanë shpërndarje më të ulët në dataset-in e plotë (**13.9%** dhe **7.5%**), por në train/test rriten ndjeshëm (**27.0%** dhe **14.7%**). Kjo zgjeron përfaqësimin e tyre dhe ndihmon modelin të mësojë tiparet dalluese të këtyre sulmeve.

3. Klasat “DoS” dhe “Reconnaissance”

Në datasetin e plotë kanë përqindje relativisht të ulët (**5.1%** dhe **4.4%**). Në train/test shënojnë rritje të dukshme (**9.9%** dhe **8.5%**), duke rritur balancën mes kategorive.

4. Klasa minoritare (Analysis, Backdoor, Shellcode, Worms)

Në të dyja setet përfaqësohen shumë pak (nën 2% secila). Megjithëse ka një rritje të vogël në train/test (p.sh. Analysis nga 0.8% në 1.6%), këto mbeten klasa të vështira për t'u trajnuar për shkak të mungesës së mjaftueshme të mostrave dhe shfaqin një sfidë për modelin. **Për shkak të peshës së ulët në set këto do të quhen klasa minoritare.**

Pjesa tjetër e analizës do të fokusohet në **dataset-et Train/Test** pasi ato do të përdoren për trajnimin e modelit. Dataset-i Train ka një numër më të lartë sulmesh të identifikuara (jo-Generic) që do të ndihmojë modelin të konvergjojë. Gjithashtu madhësia më e vogël e train/test në krahasim me setin origjinal do të ndihmoi në formimin e modeleve më komplekse.

3. 9. Analiza e Kolonave/Veçorive për Train/Test

3. 9. 1. Kontrolli i vlerave që mungojnë

Duhet të kontrollohet në qoftë se dataseti nuk përmban të dhëna të paplotësuara. Në qoftë se këto janë ekzistuese atëherë nuk mund të kryhet procesi i trajnimit. Kodi më poshtë kontrollon për çdo veçori, në qoftë se ekzistojnë **vlera që mungojnë (Vlera_NaN)** ose **objekte string bosh '' (Vlera_bosh)**. Rezultatet paraqitet ne **tabelën 3 – 2**.

```
vlera_nan = df.isna().sum()

# Konverto të gjithë hyrjet në kolona si string
# kontrollo a janë njësoj me ''
vlera_bosh = df.astype(str).apply(lambda col: col.str.strip().eq('')).sum()

rezultati = pd.DataFrame({
    'Vlera_NaN': vlera_nan,
    'Vlera_bosh': vlera_bosh
})
```

Veçoria	Vlera_NaN	Vlera_bosh
dur	0	0
proto	0	0
service	0	0
state	0	0
spkts	0	0
dpkts	0	0
sbytes	0	0
dbytes	0	0
rate	0	0
sttl	0	0
dttl	0	0
sload	0	0
dload	0	0
sloss	0	0
dloss	0	0
sinpkt	0	0
dinpkt	0	0
sjit	0	0
djit	0	0
swin	0	0
stcpb	0	0
dtcpb	0	0

Veçoria	Vlera_NaN	Vlera_bosh
tcprrt	0	0
synack	0	0
ackdat	0	0
smean	0	0
dmean	0	0
trans_depth	0	0
response_body_len	0	0
ct_srv_src	0	0
ct_state_ttl	0	0
ct_dst_ltm	0	0
ct_src_dport_ltm	0	0
ct_dst_sport_ltm	0	0
ct_dst_src_ltm	0	0
is_ftp_login	0	0
ct_ftp_cmd	0	0
ct_flw_http_mthd	0	0
ct_src_ltm	0	0
ct_srv_dst	0	0
is_sm_ips_ports	0	0
attack_cat	0	0
label	0	0

Tabela 3-2 Nurmi i vlerave NaN dhe Bosh për cdo veçori

Nuk mungon asnjë vlerë nga çdo veçori. Të dyja setet janë të plotësuar.

3. 9. 2. Vecoritë jo-numerike

Modelet ML mund të procesojnë vetëm data numerike. Në rradhë të parë duhet të zbulohen vecoritë që nuk janë të kësaj natyre dhe të enkodohen në një format numerik. Komanda **select_dtypes()** mundëson selektimin e kolonave sipas llojit, në këtë rast numerike.

```
kol_jo_numerike = df.select_dtypes(exclude=['number']).columns
kol_jo_numerike.to_list()

['proto', 'service', 'state', 'attack_cat']
```

Katër prej kolonave janë jo-numerike të tipit **objekt string**. Duhet të studiohet distribucioni i tyre për të vendosur mënyrën e duhur të enkodimit si dhe transformime të mundshme që mund të aplikohen më pas.

```
df[['proto', 'service', 'state', 'attack_cat']].dtypes

proto      object
service    object
state      object
attack_cat object
dtype: object
```

Veçoria **proto** ka 133 vlera unike. Balanca e këtyre klasave është e ulët dhe nuk ndjek një distribucion normal, aspekt i pritshëm nga të dhënat e trafiku në rrjet.

```
nr_rreshta = len(df)
# Shton kolonën për % totale
frek_proto = df['proto'].value_counts(dropna=False).to_frame('Sasia')
frek_proto['% total'] = frek_proto['Sasia'].apply(lambda x: round((x / nr_rreshta) * 100))
```

Tabela 3-3 tregon shpërndarjen e 10 protokolleve me % më të lartë dhe 10 me % më të ulët për **proto**. Duket qartë që protokollat **tcp** dhe **udp** dominojnë duke e bërë këtë veçori goxha të pabalancuar.

Tabela 3-3 % e shpërndarjes së protokolleve

10 proto me % më të lartë			10 proto me % më të ulët		
proto	Sasia	% total	proto	Sasia	% total
tcp	123041	47.75	chaos	132	0.05
udp	92701	35.98	argus	131	0.05
unas	15599	6.05	rdp	131	0.05
arp	3846	1.49	hmp	131	0.05
ospf	3271	1.27	crtp	131	0.05
sctp	1474	0.57	egp	131	0.05
any	396	0.15	netblt	131	0.05
gre	313	0.12	igmp	48	0.02
rsvp	264	0.1	icmp	15	0.01
ipv6	262	0.1	rtp	1	0

Service dhe State paraqiten nëpërmjet histrogrameve. Shihet i njëjti trend ku dominojnë 2 – 3 vlera. Për service dominojnë “-“ (nuk ka ndonjë shërbim të identifikuar për këtë rast), dns dhe http, për state FIN, INT, CON.

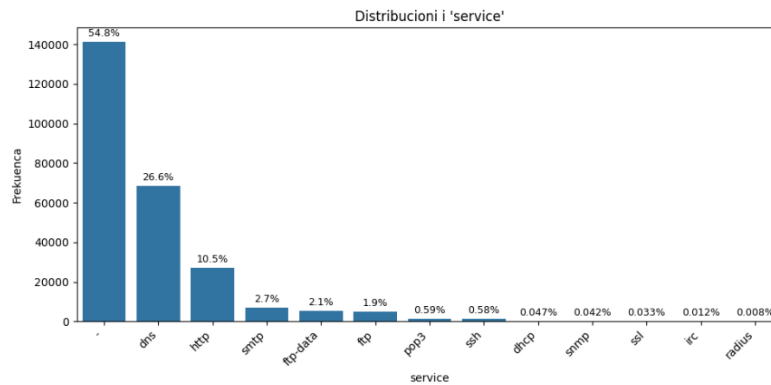


Figura 3-4 Distribucioni i veçorisë service

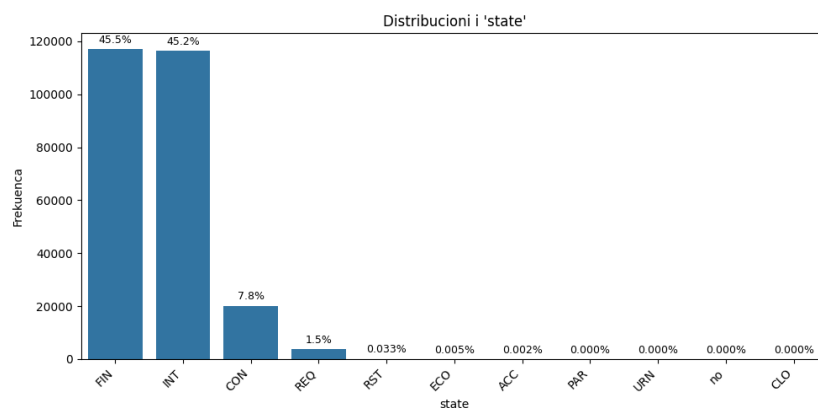


Figura 3-5 Distribucioni i veçorisë state

3. 9. 3. One-hot Encoding

Përdoret për transformimin e veçorive nga forma kategorike në vektor shumë dimensional bazuar në numrin e vlerave unike të kategorive (Géron, 2023). Formula për të llogaritur veçoritë e reja është $\sum_i^k c_i$, ku k është numri i kategorive dhe c_i është numri i veçorive unike për atë kategori. Në raste regresionesh lineare/logjistike mund të ulet numri i veçorive me 1 duke e hedhur poshtë një kategori, kjo bëhet për të shmangur multikolenaritetin (Géron, 2023). Modelet neurale nuk vuajnë nga multikolenariteti si teknikat standarte të regresionit.

Kolona service ka 13 vlera të mundshme prandaj krijohen 13 veçori të reja. Për cdo rresht vetëm 1 prej kolonave ka vlerën 1, ndërkohë që të tjerat janë 0. Kjo dikton çfarë *service* i përket.

	-	dhcp	dns	ftp	ftp-data	http	irc	pop3	radius	smtp	snmp	ssh	ssl
0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	1	0	0	0
3	0	0	0	0	0	0	0	0	0	0	1	0	0
4	0	0	0	0	0	1	0	0	0	0	0	0	0
5	0	0	0	0	1	0	0	0	0	0	0	0	0
6	0	0	1	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	0	1	0	0	0	0
9	0	0	0	0	0	0	0	1	0	0	0	0	0
10	0	1	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	1
12	0	0	0	0	0	0	1	0	0	0	0	0	0

Figura 3-6 Veçoritë e enkoduara

3. 9. 4. Enkodimi i kolonave kategorike

Kodi i mëposhtëm tregon procesin e enkodimit të të gjithë kolonave kategorike.

```

kol_per_enk = kol_jo_numerike.drop('attack_cat')
# sparse_output = False, i bënë të dhënat më të menaxhueshme
encoder = OneHotEncoder(sparse_output=False, drop=None)
data_enk = encoder.fit_transform(df[kol_per_enk])

# Merr emrat e vecorive
emra_kol = encoder.get_feature_names_out(kol_per_enk)

# Krijon Kolonat e enkoduara për cdo vecori
encoded_df = pd.DataFrame(data_enk, columns=emra_kol,
index=df.index).astype('float32')

# Fshin kolonat origjinale që u enkoduan, bashkon me data origjinale
df = pd.concat([df.drop(kol_per_enk, axis=1), encoded_df], axis=1)

```

attack_cat nuk duhet të enkodohet si vektor sepse modelet ML presin që vlera e varur/variabla që parashikohet duhet të jetë numër i plotë. Emrat e kolonave vendosen si indikator të veçorive të reja që shtohen. Numri i veçorive që shtohet llogaritet sipas formulës: $\sum_i^k c_i$.

- 'proto' : 133 vecori
- 'service' : 13 vecori
- 'state' : 11 vecori

Në total krijohen 157 kolona të reja, të enkoduara si vektor. Këtu mund të aplikohet një hap i rëndësishëm optimizimi. Funksioni **OneHotEncoding** nga Libraria Scikit-learn i ruan vlerat si float64. Çdo qelizë në dataframe është 8 byte, por në qoftë se ndryshohet tipi në float32 reduktohet në 4 byte. Reduktohet madhësia e dataset-it, nga 391.2 MB me float64 në 236.9 MB me float32:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 257673 entries, 0 to 257672
Columns: 199 entries, id to state_no
dtypes: float64(168), int64(30), object(1)
memory usage: 391.2+ MB

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 257673 entries, 0 to 257672
Columns: 199 entries, id to state_no
dtypes: float32(157), float64(11), int64(30), object(1)
memory usage: 236.9+ MB # Reduktimi i madhësisë
```

Kjo do të ndikojë pozitivisht në performancën e modelit për shkak të sasisë më të vogël të të dhënave. Vektorët e krijuar nga enkodimi paraqiten në mënyrë binare 0/1 prandaj ky konvertim nuk shkakton humbje precizioni. Kolonat origjinale *proto*, *service* dhe *state* hiqen për të shmangur konflikte gjatë procesit të trajnimit.

Veçoritë që rezultojnë nga enkodimi ndjekin strukturën e emërimit **emri_vecorisë + vlera unike**:


```
[ 'proto_3pc', 'proto_a/n', 'proto_aes-sp3-d', 'proto_any', 'proto_argus',
'proto_aris', 'proto_arp', 'proto_ax.25', 'proto_bbn-rcc', 'proto_bna', 'proto_br-
sat-mon', 'proto_cbt', 'proto_cftp', 'proto_chaos', 'proto_compaq-peer',
'proto_cphb', 'proto_cpnx', 'proto_crtp', 'proto_crudp', 'proto_dcn', 'proto_ddp',
'proto_ddx', 'proto_dgp', 'proto_egp', 'proto_eigrp', 'proto_emcon',
'proto_encap', 'proto_etherip', 'proto_fc', 'proto_fire', 'proto_ggp',
'proto_gmtp', 'proto_gre', 'proto_hmp', 'proto_i-nlsp', 'proto_iatp', 'proto_ib',
'proto_icmp', 'proto_idpr', 'proto_idpr-cmtp', 'proto_idrp', 'proto_ifmp',
'proto_igmp', 'proto_igp', 'proto_il', 'proto_ip', 'proto_ipcomp', 'proto_ipcv',
'proto_ipip', 'proto_iplt', 'proto_ipnip', 'proto_ippc', 'proto_ipv6',
'proto_ipv6-frag', 'proto_ipv6-no', 'proto_ipv6-opts', 'proto_ipv6-route',
'proto_ipx-n-ip', 'proto_irtp', 'proto_isis', 'proto_iso-ip', 'proto_iso-tp4',
'proto_kryptolan', 'proto_l2tp', 'proto_larp', 'proto_leaf-1', 'proto_leaf-2',
'proto_merit-inp', 'proto_mfe-nsp', 'proto_mhrp', 'proto_micp', 'proto_mobile',
'proto_mtp', 'proto_mux', 'proto_narp', 'proto_netblt', 'proto_nsfnet-igp',
'proto_nvp', 'proto_ospf', 'proto_pgm', 'proto_pim', 'proto_pipe', 'proto_pnni',
'proto_pri-enc', 'proto_prm', 'proto_ptp', 'proto_pup', 'proto_pvp', 'proto_qnx',
'proto_rdp', 'proto_rsvp', 'proto_rtp', 'proto_rvd', 'proto_sat-expak',
'proto_sat-mon', 'proto_sccompce', 'proto_scps', 'proto_sctp', 'proto_sdrp',
'proto_secure-vmtp', 'proto_sep', 'proto_skip', 'proto_sm', 'proto_smp',
'proto_snp', 'proto_sprite-rpc', 'proto_sps', 'proto_srp', 'proto_st2',
'proto_stp', 'proto_sun-nd', 'proto_sëipe', 'proto_tcf', 'proto_tcp',
'proto_tlsp', 'proto_tp++', 'proto_trunk-1', 'proto_trunk-2', 'proto_ttp',
'proto_udp', 'proto_unas', 'proto_uti', 'proto_vines', 'proto_visa', 'proto_vmtp',
'proto_vrrp', 'proto_ëb-expak', 'proto_ëb-mon', 'proto_ësn', 'proto_xnet',
'proto_xns-idp', 'proto_xtp', 'proto_zero',

'service_', 'service_dhcp', 'service_dns', 'service_ftp', 'service_ftp-data',
'service_http', 'service_irc', 'service_pop3', 'service_radius', 'service_smtp',
'service_snmp', 'service_ssh', 'service_ssl',

'state_ACC', 'state_CLO', 'state_CON', 'state_ECO', 'state_FIN', 'state_INT',
'state_PAR', 'state_REQ', 'state_RST', 'state_URN', 'state_no']
```

Kolonat kategorike nuk kanë nevojë për procesim të mëtejshëm.

3.9.5. Kolona Numerike

Blloku i mëposhtëm shfaqë numrin e vlerave unike për veçoritë numerike (int64 ose float64)

që ndodhen në train/test set.

```
kol_numerike = df.select_dtypes(include=np.number).columns
kol_numerike = [x for x in kol_numerike if x not in ['id', 'label']]

data_num = df[kol_numerike]
nr_unike = data_num.nunique().sort_values().reset_index()
nr_unike.columns = ['Vecoria', 'Nr_Vlera_Unike']
nr_unike['Data_Type'] = nr_unike['Vecoria'].map(data_num.dtypes)
print(nr_unike)
```

	Vecoria	Nr_Vlera_Unike	Data_Type
0	is_sm_ips_ports	2	int64
1	is_ftp_login	4	int64
2	ct_ftp_cmd	4	int64
3	ct_state_ttl	7	int64
4	dttl	9	int64
5	ct_flw_http_mthd	11	int64
6	sttl	13	int64
7	trans_depth	14	int64
8	dwin	19	int64
9	swin	22	int64
10	ct_dst_sport_ltm	35	int64
11	ct_src_ltm	52	int64
12	ct_src_dport_ltm	52	int64
13	ct_dst_ltm	52	int64
14	ct_srv_dst	57	int64
15	ct_srv_src	57	int64
16	ct_dst_src_ltm	58	int64
17	dloss	476	int64
18	sloss	490	int64
19	dpkts	627	int64
20	spkts	646	int64
21	dmean	1362	int64
22	smean	1377	int64
23	response_body_len	2819	int64
24	dbytes	8653	int64
25	sbytes	9382	int64
26	ackdat	53248	float64
27	synack	57366	float64
28	tcprtt	63878	float64
29	dur	109945	float64
30	dinpkt	110270	float64
31	dtcpb	114187	int64
32	sinpkt	114318	float64
33	stcpb	114473	int64
34	djit	114861	float64
35	rate	115763	float64
36	dload	116380	float64
37	sjit	117101	float64
38	sload	121356	float64

Këto vecori numerike duhet të evaluohen për transformim/normalizim/standartizim. **Id** shërben thjesht si index për dataframe, nuk do të përdoret gjatë trajnimit prandaj mund të hiqet pa problem. **Label** është një veçori e varur në format binar (0/1) që shërben për klasifikim kategorik dhe nuk duhet të ndryshohet.

Për të vendosur çfarë lloj ndryshimesh do aplikohen mbi kolona duhet të studiohet distribucioni i tyre. Fillimisht analizohet shpërndarja e vlerave unike për çdo veçori.

Dy veçorit me numrin më të ulët të vlerave unike **is_sm_ips_ports** dhe **is_ftp_login** janë **binare** (Moustafa, 2017) sipas dokumentimit. Një gabim është bërë gjatë përgatitjes së dataset-it dhe **is_ftp_login** përmban 2 vlera shtesë.

```
array([0, 1, 2, 4])
```

Sipas dokumentimit, vlera 1 tregon që sesioni është aksesuar nga një user dhe password, në të kundërt 0. Zëvendësohet të gjithë vlerat ilegale (2 dhe 4) me 1. (Maji, 2020)

```
data_num.loc[:, 'is_ftp_login'] = np.where(data_num['is_ftp_login'] > 1, 1,
data_num['is_ftp_login'])
data_num['is_ftp_login'].value_counts()

is_ftp_login
0      254428
1        3245
Name: count, dtype: int64
```

is_sm_ips_ports dhe **is_ftp_login** nuk kanë nevojë për procesim të mëtejshëm sepse modelet ML automatikisht janë të aftë të trajtojnë vlera binare.

KAPITULLI IV

PËRGATITJA PËR KRIJIMIN E MODELIT

Në këtë kapitull trajtohen hapat që do të përgatisin të dhënat për lexim nga modeli. Fokusi qëndron në krijimin e një pipeline dhe si përshtatet ky i fundit me ndarjen train dhe test. Në fund implementohen dhe metrikat që do përdoren për të gjykuar performancën finale të modelit.

4. 1. Çfarë është një Pipeline

Pipeline-t e librarisë scikit-learn përfaqësojnë struktura të cilat lejojnë përpunimin e të dhënave në mënyrë të strukturuar, duke ndjekur një seri hapash apo funksionesh të përcaktuara. Në këtë kontekst, shërbejnë për kryerjen e procesimit fillestar, transformimeve të nevojshme dhe shkallëzimit të variablave duke i përgatitur për modelet ML. Funksionet në pipeline përmbajnë 2 metoda standarte **.fit()** dhe **.transform()**.

.fit() – Transformuesi mëson nga të dhënat duke mbledhur vlera të nevojshme për parashikim. Nuk ndikon mbi ato origjinale. Shembuj:

- **StandartScaler()** – llogarit m (mesataren) dhe σ (devijimin standard) të veçorive.
- **OneHotEncoder()** – Definon kategoritë e mundshme brenda një veçorie kategorike (si **proto, service, state**)

.transform() – Përdor parametrat e mbledhura nga metoda **.fit()** për të aplikuar ndryshime të mundshme në data. Nuk mëson nga të dhënat.

- **StandartScaler()** – Transformon çdo veçori sipas formulës $z = \frac{x-\mu}{\sigma}$ ku x është vlera origjinale që çdo veçori të ketë μ (mesatare) = 0 dhe σ (devijim standard) = 1 .
- **OneHotEncoder()** – Rikthen veçoritë në formë kategorike si vektor multi-dimensional (1 rresht x N kolona, ku N është numri i veçorive).

4. 2. Ndarja train/test në kontekstin e modelit

Dataseti UNSW-NB15 vjen me një ndarje të paracaktuar që siguron një diversitet të madh midis llojeve të sulmeve. Siç u tregua në Figurën 3-2, dataset-i i plotë është mjaft i pabalancuar, prandaj një ndarje automatike mund të çojë në shpërndarje edhe më të pabarabartë të klasave, ekziston mundësia që disa kategori mund të mos përfshihen në test.

```
data_train = pd.read_csv('Training and Testing Sets/UNSW_NB15_training-set.csv',  
low_memory=False)  
  
data_test = pd.read_csv('Training and Testing Sets/UNSW_NB15_testing-set.csv',  
low_memory=False)
```

Të dhënat ndahen në train dhe test për të vlerësuar si dhe sa mund të performojë modeli. Pjesa e train përdoret për të mësuar rregullat – qoftë parametrat e një modeli (p.sh. peshat në një rrjet nervor) apo parametrat e transformimeve paraprake (mesatarja dhe devijimi standard te StandardScaler, λ te PowerTransformer, etj.). Pjesa **test** mbetet “e paprekur” gjatë këtij procesi, dhe shërben si subjekt për performancën e modelit, duke funksionuar si të dhëna të reja. Kjo ndarje është kritike në projekte si IDS sepse simulon trafikun e panjohur që vjen në rrjet.

4. 2. 1. Aplikimi i .fit() dhe .transform()

Kur aplikohen transformime si **StandardScaler()** apo **OneHotEncoding()**, ekziston një dallim i qartë midis **.fit()** dhe **.transform()**. Në train set, **.fit()** përdoret për të llogaritur parametrat nga të dhënat dhe më pas **.transform()** për ti transformuar ato sipas këtyre parametrave që u mësuan. Scikit-learn përmban funksionin **fit_transform()** që i aplikon të 2 proceset. Në test set nuk aplikohet **.fit()**, përdoret vetëm **.transform()** me parametrat e ruajtur nga train. Ky proces garanton që transformimet të jenë të njëjta për të dy grupet dhe **shmang rrjedhjen e informacionit nga train në test (fenomeni ku modeli mëson dhe nga seti i test)**. Për shembull, nëse **StandardScaler()** mëson mesataren e **ct_dst_ltm** nga train, e njëjta mesatare përdoret për të standardizuar edhe test, pavarësisht nëse shpërndarja e test është ndryshe.

Në qoftë se modeli vuan nga rrjedhja e informacionit, ai do të mësojë dhe nga të dhënat e test ku do shfaqet fenomeni i **Overfitting**. Në këtë rast modeli mëson dhe nga data të test, gjë që jep përshtypjen e performancës së mirë (Shembuj: Saktësi gati absolute, pak humbje gjatë trajnimit), por në momentin që shfaqen të dhëna të reja modeli nuk performon.

4. 2. 2. Ndarja e seteve në komponent X dhe y

Të dy setet train dhe test do të ndahen më tej në 2 komponentë **X** dhe **y**.

X – vlerat e pavarura input (hyrëse) që përbehen nga të gjithë kolonat e modelit që do të përdoren për parashikim. Praktikisht këto janë të dhënat e trafikut.

y – vlera e varur target(objektiv), që përbehet nga veçoria **attack_cat**. Kjo është veçoria që modeli do të parashikojë nga X. Vlerat në këtë kolonë janë objekte string, dhe kanë nevojë për enkodim që të paraqiten si numra të plotë. Ky proces trajtohet mbas shpjegimit të pipeline.

Si përfundim do kemi 4 sete:

- X_train, X_test
- y_train, y_test

Për këtë model do hiqen kolonat e veçorive kategorike dhe kolona **attack_cat**. Lista e krijuar do të përdoret si përcaktues se te cilat kolona do të aplikohen operacionet.

```
df = df.drop(['id', 'proto', 'service', 'state', 'label'], axis=1)
kol = df.columns.to_list()
kol.remove('attack_cat')
```

```

ct_Pipeline = Pipeline(steps=[
    ('scale', StandardScaler())
])

preprocessor = ColumnTransformer(
    transformers=[
        ('col', ct_Pipeline, kol)
    ],
    remainder='drop',
    verbose_feature_names_out=False
).set_output(transform='pandas')

```

Sipër është një pipeline i thjeshtë që përmban vetëm një hap, **'scale'**, që standardizon vlerat prezente në një veçori. Shkallëzimi ndihmon në datase të diverse me veçori që kanë njësi të ndryshme. Veçoritë **ct_ (count)** që përbëhen me numra të plotë, *Sintpkt/Dintpkt* që maten me **msec** si dhe binare si *is_ftp_login, is_sm_ips*. Modelet Neurale punojnë më mirë me të dhëna që kanë shkallë të përafërta. Shkallëzimi unifikon shpërndarjen e të dhënave duke përmirësuar mundësinë e konvergimit të modelit.

ColumnTransformer() lejon aplikimin e transformimeve të definuara në pipeline në kolona specifike. Në këtë rast çdo kolonë numerike do standardizohet, kalohet si parametër lista **kol** që përmban cdo vecori përveçse **attack_cat**. Output-i i pipeline do të jetë një dataframe nga pandas që përmban vetëm kolonat e specifikuar.

```

x_train = preprocessor.fit_transform(data_train)
x_test = preprocessor.transform(data_test)

```

Siç u shpjegua më parë, për të dhëna test aplikojmë vetëm **transform()**.

4. 3. Enkodimi i **attack_cat**

Kolona **attack_cat** është një veçori kategorike, prandaj duhet konvertuar në formë numerike që të kuptohet nga modeli. **LabelEncoding()** transformon të dhëna string në numra të plotë, ndryshe nga **OneHotEncoder()** që kthen vektor multidimensional.

```

LabelEnc = LabelEncoder()
y_train = LabelEnc.fit_transform(data_train['attack_cat'])
y_test = LabelEnc.transform(data_test['attack_cat'])

for klasa, vlera in zip(LabelEnc.classes_, range(len(LabelEnc.classes_))):
    print(f"{klasa} → {vlera}")

```

```

Analysis          → 0
Backdoor          → 1
DoS               → 2
Exploits          → 3
Fuzzers           → 4
Generic           → 5
Normal            → 6
Reconnaissance    → 7
Shellcode         → 8
Worms             → 9

```

Njësoj si të dhënat e setit X .fit() aplikohet vetëm te të dhënat train dhe jo te seti test.

4. 4. Peshat e Klasave

Për shkak se dataseti UNSW-NB15 është i pa balancuar është praktikë efektive implementimi i peshave specifike për klasat. Secilës klasë i vendoset një peshë në proporcion të zhdrejtë me madhësinë e klasës. Trafiku *Normal* (56000 raste) do ketë një peshë të ulët ndërsa *Worms* do kenë peshën më të lartë (vetëm 130 raste). Në këtë mënyrë influencohet modeli mos të injorojë klasat që e vogla në dataset. Kjo ndihmon në rritjen e **recall** për sulmet e rralla.

Formula për llogaritjen e peshave është
$$\frac{\text{numri total i rreshtave}}{(\text{nr.klasave} * \text{nr.rreshtave per klasë})}$$

Këto pesha i kalohen modelit gjatë trajnimit. Nëse modeli keq-klasifikon një rast që aktualisht ishte *Worms* penalizohet më shumë sesa po të gabonte për klasën *Normal*. Kjo qasje ndihmon në përmirësimin e balancës mes precision/recall për klasat minoritare dhe e bën IDS-in të jetë më i ndjeshëm ndaj sulmeve të rralla.

```

n_classes = len(LabelEnc.classes_)
classes = np.arange(n_classes)
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,

```



```
y=y_train  
)
```

`class_weight='balanced'` siguron që shkallëzimi i peshës midis klasave është linear.

0, Analysis:	8.767
1, Backdoor:	10.042
2, DoS:	1.43
3, Exploits:	0.525
4, Fuzzers:	0.964
5, Generic:	0.438
6, Normal:	0.313
7, Reconnaissance:	1.671
8, Shellcode:	15.476
9, Worms:	134.878

4. 5. Metrikat e performancës

Implementohen metrika vlerësimi për të gjykuar performancën e modeleve si dhe efektivitetin e teknikave të optimizimit që do të zhvillohen:

Accuracy(Saktësia e përgjithshme): Mat saktësinë e parashikimeve për rastet pozitive bashkë me ato negative.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- TP (Pozitivë të saktë): Rastet pozitive dhe modeli i parashikoi si pozitive.
- TN (Negativë të saktë): Rastet që ishin negative dhe modeli i parashikoi si negative.
- FP (Gabime pozitive): Rastet që ishin negative por modeli i parashikoi si pozitive.
- FN (Gabime negative): Rastet që ishin pozitive por modeli i parashikoi si negative, nuk i kapi.

Accuracy në vetvete nuk është metrikë efektive për dataset-e të pa balancuara. Në qoftë se 95% e data është trafik normal, dhe vetëm 5% janë sulme modeli mund të ketë saktësi të përgjithshme 95%, thjesht duke klasifikuar çdo paketë si normale. Ky problem thellohet më

shumë n.q.s. hyjmë në përcaktimin e klasave individuale prezentëf në kategorinë e sulmeve.

Për këtë arsye përdoren metrika të tjera saktësie që përcaktohen më poshtë:

Precision (Saktësia): Tregon se sa prej klasave që u përcaktuan si pozitive janë aktualisht pozitive. Nga rastet e vërteta sa prej tyre u kapën. Nëse modeli thotë se 100 paketa janë sulm dhe 80 prej tyre janë realisht sulm, atëherë $\text{precision} = 0.8$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{80}{80 + 20}$$

Recall (Ndjeshmëria): Sa prej rasteve që janë pozitive janë parashikuar si të tilla nga modeli. Një recall i lartë shpreh që modeli humb shumë pak raste. Llogaritet nëpërmjet raportit të të vërtetave positive me të vërtetat pozitive plus gabimet negative. Nëse në total ka 100 paketa sulmi dhe modeli arrin të zbulojë vetëm 70 prej tyre, atëherë $\text{recall} = 70\%$.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{70}{70 + 30}$$

F1-Score: Është mesatarja harmonike mes precision dhe recall. Sa më afër 1 të jetë F1, aq më i ekuilibruar dhe i mirë është modeli në të dyja drejtimet.

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Në këtë rast recall më i lartë është me i rëndësishëm sesa precision sepse duhet të kapim gabimet e mundshme. Duhet patur parasysh që ekziston një këmbim mes Precision dhe Recall. Shumica e modeleve të parashikimit varen nga kufij probabilitar, (nëse $x > 0$ e vërtetë në të kundërt x është i gabuar), prandaj është e rëndësishme që modeli të optimizohet sipas rastit të përdorimit. Në rastin e IDS paketat e dëmshme të trafikut janë më të kushtueshme prandaj modeli duhet të optimizohet për të pasur Recall të lartë (nga të gjithë paketat realisht të dëmshme sa prej tyre u kapën nga modeli). Precision optimizon për të shmangur gabimet e mundshëm, të cilat janë me prioritet më të ulët për IDS.

KAPITULLI V

IMPLEMENTIMI DHE TESTIMI I MODELEVE

5. 1. Struktura e Modelit

Kodi poshtë përcakton strukturën e modelit, ku çdo rresht tregon një shtresë individuale individuale:

```
inputs = tf.keras.Input(shape=(x_train.shape[1],), name='features')
x = tf.keras.layers.Dense(128, activation='relu')(inputs)
x = tf.keras.layers.Dropout(0.3)(x)
x = tf.keras.layers.Dense(64, activation='relu')(x)
outputs = tf.keras.layers.Dense(n_classes, activation='softmax',
name='attack_cat')(x)
```

Shtresa 1 përgatit variablin **inputs**, i cili merr çdo rresht që ndodhet në setin train dhe ia kalon modelit në formën e një vektori multidimensional me formën (1x39), numri i kolonave është më i ulët sepse fillimisht do testohen vetëm veçoritë numerike.

Shtresa 2 Dense(128) inputs shërben si variabël hyrës për procesimin. Kjo shtresë përbëhet nga 128 neurone me kusht aktivizimi **ReLU (Rectified Linear Unit)** për të mundësuar mësimin nga data jo-lineare. Formula e llogaritjes së vlerës të një neuroni **j** është:

$$z_j = \sum_{i=1}^n w_{ij} * x_i + b_j$$

- $n \rightarrow$ është numri i veçorive
- $i \rightarrow$ indeksi i veçorive
- $j \rightarrow$ indeksi i neuronit që po procesohet
- $x_i \rightarrow$ veçoria e i -të
- $w_{ij} \rightarrow$ pesha që e lidh me neuronin
- $b_j \rightarrow$ bias për neuronin
- $z_j \rightarrow$ rezultati i neuronit para aplikimit të funksionit të aktivizimit

$$f(z_j) = \max(0, z_j)$$

- Nëse $z_j > 0 = z_j$
- Nëse $z_j \leq 0 = z_j$

Ky funksion përdoret gjerësisht në ML për shkak të natyrës së thjeshtë. Kjo shtresë shërben për të nxjerrë tipare të kombinuara nga inputet e IDS-it (Si lidhen veçoritë me njëra-tjetrën). Nëse z_j ka vlerë pozitive atëherë aktivizohet neuroni, në të kundërt jo. Kështu neuronet mund të diversifikohen dhe reagojnë ndaj veçorive të ndryshme. **Output-i do të jetë një vektor multidimensional me gjatësinë e batch_size (numri i rasteve që procesohen në të njëjtën kohë) dhe 128 kolona.**

Shtresa 3 Dropout() fik një % të specifikuar të neuroneve të cilat u aktivizuan në shtresën paraardhëse. Në këtë rast 30%. Ky proces rrit diversitetin e rrjetit neural duke e forcuar modelin të përdori kombinime të ndryshme neuronesh. **Kjo gjithashtu ul mundësitë e Overfitting** duke forcuar neuronet të bashkëveprojnë dhe duke e bërë modelin më të aftë kundrejt datave të reja. Ky proces nuk ndryshon dimensionet e vektorit.

Shtresa 4 është një tjetër Dense() me aktivizim ReLU por në këtë rast me 64 neurone. Kjo bëhet për të përmbledhur në mënyrë graduale rezultatet e modelit në vektorë vazhdimisht më të vegjël, duke kombinuar rezultatet e hapave të mëparshëm në një mënyrë më të thellë. Ky reduktim gradual i dimensioneve vazhdon derisa të kemi vektor multidimensional me **batch_size x 10 (nr. i llojeve të trafikut) dimensione.**

Shtresa 5 Output, Një tjetër shtresë Dense(), por në këtë rast përmban n_classes neurone (10, sa numri i kategorive të sulmeve). Softmax e kthen këtë input në një vektor distribucioni probabilistik me të njëjtat dimensionet si vektori input:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Input:	[2, 1, 0.1]
Softmax:	[0.66, 0.24, 0.1]

```
Shuma e prob: 1.0
```

Përcaktimi i klasës së trafikut varet nga kushti që përcakton funksioni i aktivizimit të neuroneve. **Argmax** zgjedh klasën me probabilitetin më të lartë. Pra në këtë rast atë që është 65%. Një funksion më kompleks aktivizimi mund të kryejë një analizë më të thellë.

Kodi poshtë deklaron objektin **model**. Merr parametër hyrës inputs dhe strukturën e rezultateve në outputs.

```
model = tf.keras.Model(inputs, outputs)
model.summary()
```

Figura 5-1 shpreh strukturën e modelit. Paraqet çdo shtresë si dhe numrin e parametrave që përmbajnë. Formula për të llogaritur numrin e parametrave në një shtresë **Dense()** është:

$$nr.param = (n_{in} * n_{out}) + n_{out}$$

ku: n_{in} është numri i veçorive input, n_{out} numri i veçorive output. n_{out} i dytë shpreh bias për neuronin. Shtresa 1 $nr.param = (39 * 128) + 128 = 5120$.

Layer (type)	Output Shape	Param #
features (InputLayer)	(None , 39)	0
dense (Dense)	(None , 128)	5,120
dropout (Dropout)	(None , 128)	0
dense_1 (Dense)	(None , 64)	8,256
attack_cat (Dense)	(None , 10)	650

Figura 5-1 Struktura e modelit bazik

5. 1. 1. Specifikime për ekzekutimin

Funksioni `.compile()` përcakton disa specifikime për modelin që do referohen gjatë procesit të trajnimit.

```
model.compile(  
    optimizer = 'adam',  
    loss = 'sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Loss është funksioni i humbjes i cili llogarit nivelin e gabimit të modelit duke krahasuar vlerat e parashikuara me ato reale. Në këtë rast përdoret ‘**sparse_categorical_crossentropy**’. Ky funksion është efektiv për modele ku output-i përbehet nga disa klasa dhe përdoret paralelisht me `LabelEncoding()`, si në rastin e kategorizimit të trafikut. Ky funksion merr vlerën më të lartë probabilitare nga vektori i gjeneruar nga **softmax** dhe aplikon funksionin:

$$loss = -\log(p_y)$$

ku p_y është probabiliteti më i lartë në vektor. Sa më e ulët vlera e funksionit **Loss** aq më lartë performanca e modelit. **Me rritjen e probabilitetit të variablit y ulet vlera e funksionit Loss që tregon përmirsim të modelit.**

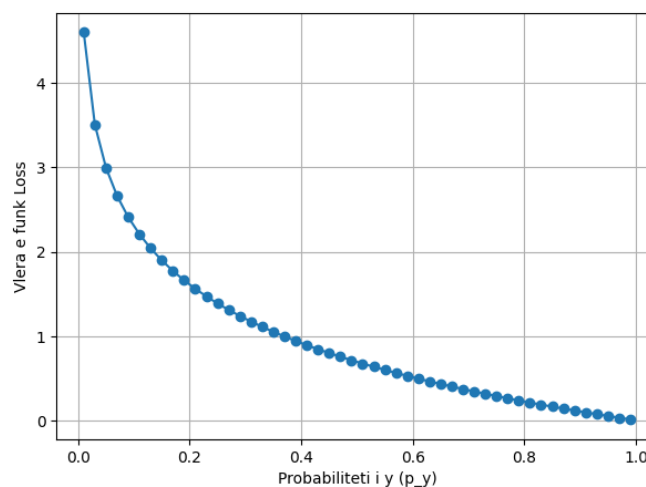


Figura 5-2 Paraqitja e funksionit. Loss nëpër epoka

Optimizuesi Adam është një algoritëm që përdoret për të minimizuar vlerën e funksionit Loss përgjatë epokave të trajnimit. E bën këtë duke përditësuar peshat e rrjetit nervor në çdo hap trajnimi, bazuar në gradientët që tregojnë drejtimin e humbjeve në rrjet. Implementimi i këtij algoritmi nuk do të eksplorohej më tej për shkak se bie jashtë fushës së tezës.

Learning rate (shkalla e mësimi) është një hiperparametër (përcaktohet brenda objektit **Adam(learning_rate=x)**, vlera standarte është 0.001) që përcakton madhësinë e hapave që modeli bën kur përditëson peshat gjatë trajnimit. Një vlerë shumë e lartë e learning rate mund të shkaktojë lëkundje ose divergjencë, ndërsa një vlerë shumë e ulët e bën procesin e mësimi shumë të ngadaltë dhe rrezikon ngulitje në minimume lokale (loss ngel e njejtë gjatë epokave).

Avantazhe të optimizuesit Adam për dataset IDS:

- Performancë e lartë me të dhëna shumë-dimensionale – **Adam** përshtat **learning rate** individualisht për çdo parametër (nuk e ndryshon vlerën standarde, por e rregullon në mënyrë dinamike).
- Efektiv në dataset-e me shumë zhurmë ose klasa të pabalancuara – UNSW-NB15 dhe dataset-e të tjera të trafikut të rrjetit vuajnë nga këto karakteristika.
- Nevojë e ulët për akordim manual – parametrat e tij të paracaktuar japin rezultate të mira në shumicën e rasteve.

Accuracy(Saktësia e përgjithshme) është specifikuar në **.compile()**, do të paraqitet gjatë procesit të trajnimit. Për të implementuar metrikat e tjera të specifikuara më sipër do të përdoret funksioni **classification_report** nga libraria **scikit-learn**.

5. 1. 2. Fillimi i trajnimit të modelit bazik

model.fit() fillon trajnimin e modelit. Objekti **history** ruan të dhëna në lidhje me këtë proces që mund të analizohen pas përfundimit të trajnimit.

```
history = model.fit(  
    x_train, y_train,  
    validation_split=0.1,  
    epochs=60,  
    batch_size=1024,  
    class_weight=class_weights_dict,  
    verbose=1  
)
```

Epoka (Epochs) dikton sa herë modeli do procesojë tërë të dhënat që ndodhet në **x_train** dhe **y_train**. Në këtë rast 60 herë. Mbas çdo kalimi të plotë modeli rregullon peshat dhe vlerëson performancën. Me implementimin e algoritmeve **EarlyStopping()** mund të monitorohet kjo dhe të rikthehen peshat më efektive për modelin.

Për këtë ndihmon parametri **validation_split**. Merr një % të specifikuar të data për trajnim dhe e përdor për të evaluar modelin në fund të epokës. **Në qoftë se val_accuracy dhe val_loss nuk tregojnë përmirësim (rritje për accuracy, ulje për loss) tregon që modeli nuk u përmirësua gjatë kësaj epoke.** Nëse ky trend vazhdon ose ka rënie të vlerave val atëherë modeli vuan nga **overfitting** (modeli mëson shumë sipas të dhënave të train dhe vuan kundrejt atyre që nuk i ka hasur më përpara, ato në setin test).

Batch_size specifikon sa rreshta (raste) modeli përpunon në të njëjtën kohë. Ky parametër ndikon në kohën e nevojshme për trajnimin e modelit. Sa më e lartë **batch_size** aq më i shpejtë procesi. Gradientët gjithashtu ndikohen nga **batch_size**. Një vlerë shumë e lartë për **batch_size** e përditëson shumë ngadalë gradientin gjë që ngadalëson procesin e mësimin, ndërsa një vlerë shumë e ulët shton zhurmë dhe ndikon negativisht në metrikat e tij. Pas testimeve me modelin e sipërm u vendos që **batch_size= 1024** është optimale për performancën e modelit.

Madhësia e vogël e datasetit (90.4 MB) dhe numri relativisht i ulët i veçorive lejon përdorimin e vlerave të larta **batch_size** pa shqetësimin e limiteve në VRAM.

5. 1. 3. Procesi i trajnimit të modelit bazik

Poshtë tregohet procesi i trajnimit të modelit sipas epokave në **Jupyter Notebook**. Trajnimi ka zgjatur për 60 epoka, specifikuar në **model.fit()**. Analizimi i vlerës Loss dhe Accuracy nëpër epoka nuk tregon përmirësim të dukshëm, as në epokat fillestare, ku zakonisht modeli përmirësohet me shpejtësi më të lartë. Modeli po qëndron në një stad stagnant.

```
Epoch 1/60
155/155 ————— 5s 19ms/step - accuracy: 0.7218 - loss: 0.7880 - val_accuracy: 0.8997 - val_loss: 0.2922
Epoch 2/60
155/155 ————— 1s 4ms/step - accuracy: 0.7229 - loss: 0.7871 - val_accuracy: 0.8967 - val_loss: 0.2971
Epoch 3/60
155/155 ————— 0s 3ms/step - accuracy: 0.7230 - loss: 0.7806 - val_accuracy: 0.9058 - val_loss: 0.2933
Epoch 4/60
155/155 ————— 0s 3ms/step - accuracy: 0.7247 - loss: 0.7705 - val_accuracy: 0.9025 - val_loss: 0.2901
Epoch 5/60
155/155 ————— 0s 3ms/step - accuracy: 0.7236 - loss: 0.7734 - val_accuracy: 0.9068 - val_loss: 0.2895
Epoch 6/60
155/155 ————— 0s 3ms/step - accuracy: 0.7202 - loss: 0.7767 - val_accuracy: 0.9023 - val_loss: 0.2941
Epoch 7/60
155/155 ————— 0s 3ms/step - accuracy: 0.7202 - loss: 0.7715 - val_accuracy: 0.9058 - val_loss: 0.2900
Epoch 8/60
155/155 ————— 0s 3ms/step - accuracy: 0.7254 - loss: 0.7676 - val_accuracy: 0.9052 - val_loss: 0.2840
Epoch 9/60
155/155 ————— 0s 3ms/step - accuracy: 0.7209 - loss: 0.7839 - val_accuracy: 0.9009 - val_loss: 0.2850
Epoch 10/60
155/155 ————— 0s 3ms/step - accuracy: 0.7221 - loss: 0.7795 - val_accuracy: 0.8981 - val_loss: 0.2906
Epoch 11/60
155/155 ————— 1s 3ms/step - accuracy: 0.7210 - loss: 0.7783 - val_accuracy: 0.9008 - val_loss: 0.2919
Epoch 12/60
155/155 ————— 1s 3ms/step - accuracy: 0.7230 - loss: 0.7848 - val_accuracy: 0.9039 - val_loss: 0.2953
Epoch 13/60
...
Epoch 59/60
155/155 ————— 0s 3ms/step - accuracy: 0.7226 - loss: 0.7855 - val_accuracy: 0.9001 - val_loss: 0.2952
Epoch 60/60
155/155 ————— 0s 2ms/step - accuracy: 0.7263 - loss: 0.7838 - val_accuracy: 0.8975 - val_loss: 0.2921
```

Figura 5-3 Progresi i modelit përgjatë epokave

Grafiku i mëposhtëm shpreh si ka lëvizur vlera e **Loss** përgjatë trajnimit. Vija blu tregon dataset-in aktual train dhe ajo portokalli të validimit. Duke parë formën shohim që **Loss** në fakt rritet sa më shumë epoka kalojnë, kur në fakt duhet të ulej më tej. Kjo tregon që modeli nuk po performon në mënyrë optimale e duhet të kryhen optimizime të mëtejshme.

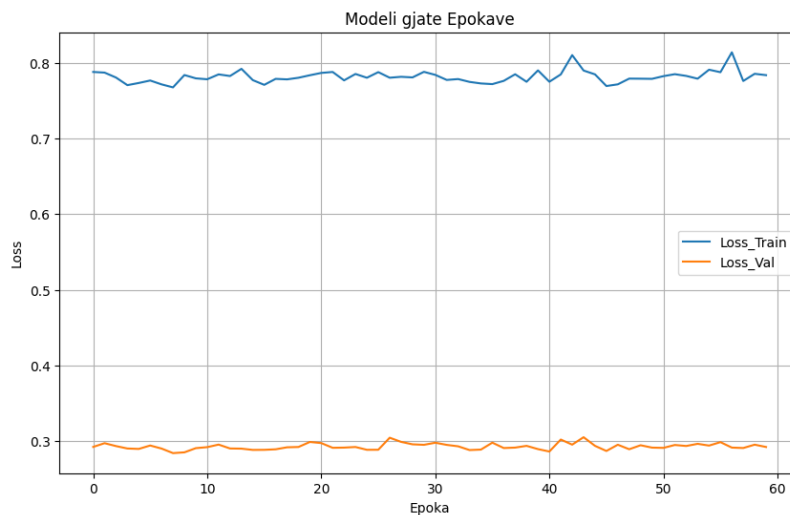


Figura 5-4 Loss e modelit përgjatë epokave

Grafiku tjetër shpreh nivelin e Saktësisë së përgjithshme gjatë epokave. Ka pak varietet midis vlerave por në përgjithësi qëndron konstant. Pohon rezultatin në fund të paragrafit të mësipërm.

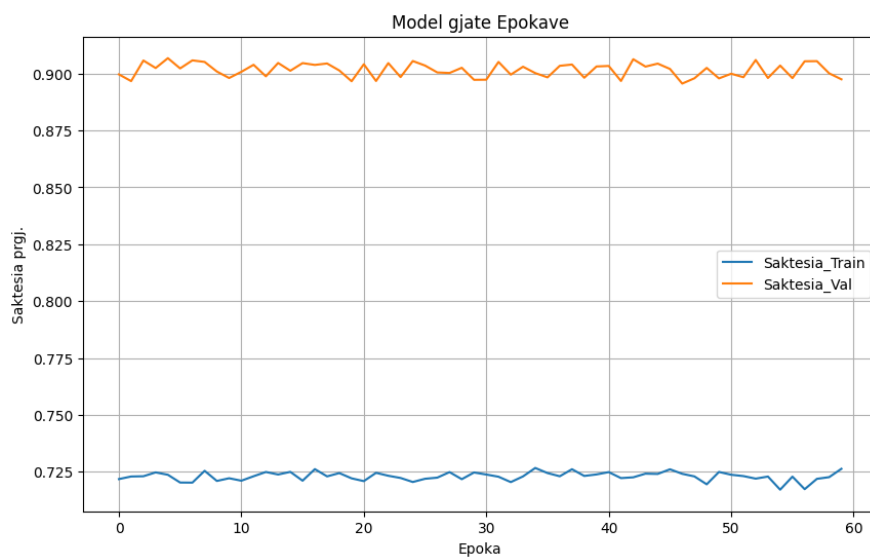


Figura 5-5 Saktësia e modelit përgjatë epokave

Diferenca e madhe mes vlerave të **Loss** dhe **Acc** përmes setit të trajnimit dhe atij të validimit vjen për shkak se ky i fundit nuk llogaritet në bazë të peshave të klasave. Gjatë trajnimit modeli penalizohet rëndë nga parashikimet e gabuara kundrejt klasave minoritare për shkak të peshave më të larta. Gjithsesi, accuracy nuk është metrikë e mirë për dataset-e të pabalancuara.

5. 1. 4. Interpretimi i rezultateve për modelin bazik

Tabela e mëposhtme shpreh metrikat e **precision**, **recall** dhe **f1-score** për secilën nga kategoritë e mundshme të trafikut.

Kategoria	precision	recall	f1-score	support
Analysis	0	0.01	0	677
Backdoor	0.06	0.77	0.11	583
DoS	0.36	0.22	0.27	4089
Exploits	0.83	0.51	0.63	11132
Fuzzers	0.24	0.63	0.35	6062
Generic	1	0.96	0.98	18871
Normal	1	0.58	0.74	37000
Reconnaissance	0.82	0.84	0.83	3496
Shellcode	0.12	0.93	0.21	378
Worms	0.03	0.86	0.06	44
accuracy			0.65	82332
macro avg	0.45	0.63	0.42	82332
weighted avg	0.86	0.65	0.72	82332

Tabela 5-1 Performanca e modelit bazik

Kolona **support** jep numrin e rasteve për kategorinë **X** në setin test. Shihet që dy klasat mazhorante *Normal* dhe *Generic* kanë **precision** = 1, parashikimet e modelit janë gjithmonë korrekte por, recall i *Normal* tregon që modeli ka humbur **42%(1-0.58)** të trafikut normal.

Një pjesë e sulmeve që janë pjesë e kategorive minoritare *Backdoor*, *Shellcode*, *Worms* kanë **vlera të larta recall-i (0.77, 0.93, 0.86)** por **precizion shumë të dobët (0.06, 0.12, 0.03)**. Modeli është i suksesshëm në klasifikimin e rasteve reale por bën dhe shumë gabime duke keq-klasifikuar lloje të tjera trafiku (**FP**). Kjo vjen për shkak se modeli favorizon shumë klasat minoritare për shkak të implementimit të peshave bazuar në raporte lineare.

Reconnaissance shfaq vlera të kënaqshme precizioni (0.82) dhe recall (0.93). Kjo tregon që për këtë sulm modeli prioritetizon si duhet tolerimin e FP për të minimizuar FN.

Por ka raste kur modeli injoron plotësisht kategori minoritare si **Analysis**. Precision dhe recall janë praktikisht 0. Kjo me shumë mundësi rezulton për shkak të heqjes nga testi të veçorive [proto, service, state], që do të shtohen në modelin tjetër.

Confusion Matrix e bënë më të lehtë analizimin e distribucionit të vlerave të parashikuara, boshti x, me ato reale, boshti y. Diagonalja nga majtas lart → djathas poshtë tregon recall. Kolonat shprehin % e klasës reale (boshti y) që është klasifikuar si një klasë tjetër.

Vlera Reale	Analysis	0.6%	73.1%	20.1%	4.3%	1.8%	0.0%	0.1%	0.0%	0.0%	0.0%
	Backdoor	0.7%	76.8%	19.4%	0.9%	0.2%	0.0%	0.0%	0.3%	1.4%	0.3%
	DoS	1.3%	56.4%	22.2%	11.6%	2.0%	0.1%	0.0%	0.8%	3.0%	2.6%
	Exploits	1.1%	23.8%	8.3%	50.6%	2.9%	0.0%	0.1%	3.7%	3.7%	5.8%
	Fuzzers	0.1%	17.0%	4.1%	0.6%	63.1%	0.0%	0.4%	1.0%	11.1%	2.7%
	Generic	0.0%	0.3%	0.6%	1.7%	0.4%	96.2%	0.0%	0.0%	0.2%	0.7%
	Normal	5.8%	0.1%	0.1%	0.8%	31.1%	0.0%	58.2%	0.3%	3.2%	0.4%
	Reconnaissance	0.1%	7.6%	2.1%	0.7%	0.4%	0.0%	0.0%	84.2%	3.3%	1.5%
	Shellcode	0.0%	0.3%	0.0%	0.0%	0.8%	0.0%	0.3%	3.7%	93.4%	1.6%
	Worms	0.0%	0.0%	0.0%	0.0%	2.3%	0.0%	0.0%	0.0%	11.4%	86.4%
		Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Reconnaissance	Shellcode	Worms
Vlera e Parashikuar											

Figura 5-6 Matrica për modelin bazik

Si shembull 0.4% e trafikut të **Normal** dhe 0.7% e trafikut të **Generic** janë klasifikuar gabimisht si **Worms**. Kjo e bën precisionin të bjerë pasi këto janë klasa mazhorene. **Worm** ka vetëm 44 raste në setin test. $(0.004 * 37000 + 0.007 * 18871) \approx 280$ raste të klasifikuara gabimisht si **Worm**.

$Precision = \frac{TP}{TP+FP} = \frac{38}{38+280} \approx 0.119$ (Kjo vlerë u llogarit me vetëm 2 klasa në qoftë se shtojmë dhe keq-klasifikimet e tjera do bëhet e barabartë me vlerën e tabelës). **Backdoor** shfaq

një sjellje të njëjtë por shihet qartë që po kanibalizon klasat e **Analysis (73.1%)** dhe **Dos (56.4%)**. Pjesa më e madhe e trafikut që i përket këtyre klasave po klasifikohet si **Backdoor**.

Dhe pse në pamje të parë performanca mund të duket e dobët ky është një rrjet neural me vetëm 5 shtresa që u trajnuar për rreth 2 minuta. Performanca për klasa mazhorante është e pranueshme dhe detekton një pjesë të mirë të sulmeve. Problem kryesor janë numri i lartë i **FP** (Trafiku Normal që klasifikohet si sulm) që rezultojnë nga recall i ulët i klasës **Normal** dhe keq-klasifikimi midis Sulmeve në klasa minoritare (Precizion i ulët).

Analiza e mëtejshme do vazhdojë mbas shtimit të optimizimeve të reja të pipeline.

5. 2. Implementimi i OneHotEncoder

5. 2. 1. One Hot Encoder

Shtohet një hap i ri për në pipeline. Definohet si **OneHot** dhe thirret funksioni. Parametri `handle_unknown='ignore'` menaxhon rastin kur gjatë testit haset një kategori që nuk u procesua gjatë `.fit()` për setin test, duke krijuar një vektor vetëm me 0.

```
kat_Pipeline = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
preprocessor = ColumnTransformer(
    transformers=[
        ('col', ct_Pipeline, kol_num),
        ('cat', kat_Pipeline, kol_kat)
    ],
    remainder='drop',
    verbose_feature_names_out=False
).set_output(transform='pandas')
```

Praktikë e mirë është kontrollimi i dimensioneve të seteve që krijohen. Funksioni `len(preprocessor.get_feature_names_out())` kthen vlerën 194, por sipas llogarive **39 kolona numerike + kolona për enkodimi (133 proto + 13 service + 11 state) = 196**. Disa kategori janë unike për një nga setet (me blu). Paraqiten në bllokun e mëposhtëm:

```

Unike per test proto: set()
Unike per train proto: {'rtp', 'icmp'}

Unike per test service: set()
Unike per train service: set()

Unike per test state: {'ACC', 'CLO'}
Unike per train state: {'PAR', 'no', 'ECO', 'URN'}

```

Tani duket qartë që setet kanë disa diferenca në protokolle. Në këtë rast objekti për enkodimin duhet të krijohej manualisht. **kat_vocab** përmban të ndara në lista vlerat unike për secilën nga 3 veçoritë kategorike. Kalohet si parametër i ri tek objekti **ohe**.

```

kat_vocab = []
for kol in kol_kat:
    vocab = sorted(set(data_train[kol].unique()) | set(data_test[kol].unique()))
    kat_vocab.append(vocab)

total_kat_kol = sum(len(v) for v in kat_vocab)

ohe = OneHotEncoder(categories=kat_vocab,
                    handle_unknown='ignore',
                    sparse_output=False)

```

Në këtë mënyrë sigurohet që të dy setet e trajnimit dhe testimit enkodohen nga një objekt i cili përfshin klasat e të dy seteve. Hapi i fundit është shtimi i pipeline të ri te seksioni i transformuesve.

```

preprocessor = ColumnTransformer(
    transformers=[
        ('col', ct_Pipeline, kol_num),
        ('cat', kat_Pipeline, kol_kat)
    ],

Forma e train: (175341, 196) # Dimensionet në rregull
Forma e test: (82332, 196) # Dimensionet në rregull

```

5. 2. 2. Interpretimi i Modelit OHE

Me shtimin e OneHot Encoding, *Analysis* ka filluar të kapet nga modeli. Gjithsesi metrikat nuk janë të kënaqshme. **Recall** 0.07 tregon që modeli është i aftë të klasifikojë vetëm 7% të rasteve të vërteta. Varianca midis vlerave të tjera është thjesht rezultat i trajnimit të modelit nga e para dhe natyrës rastësore së modeleve neurale.

Kategoria	precision	recall	f1-score	support
Analysis	0.02	0.07	0.03	677
Backdoor	0.06	0.71	0.11	583
DoS	0.38	0.21	0.27	4089
Exploits	0.86	0.48	0.62	11132
Fuzzers	0.25	0.66	0.36	6062
Generic	1	0.96	0.98	18871
Normal	1	0.58	0.73	37000
Reconnaissance	0.63	0.83	0.72	3496
Shellcode	0.13	0.81	0.22	378
Worms	0.02	0.82	0.04	44

Tabela 5-2 Performanca e modelit me OHE

Ka një hap tjetër **optimizimi** që mund të aplikohet në lidhje me procesin e trajnimit. Vlera e loss nga validimi është shumë më e ulët se ajo që rezulton gjatë trajnimit (Figura 5-4). Ky problem haset shpesh në datasete të pabalancuara si UNSW-NB15. Seti i val nuk po përshin data nga të gjithë kategoritë të sulmeve të mundshme.

OHE nuk aplikohet mbi të dhëna kardinale (raste kur një numër më i madh tregon më shumë) pasi efektivisht i transformon këto veçori në binare duke zhdukur këtë informacion që në të kundërt mund të ishte i dobishëm për modelin. Praktikisht çdo veçori që fillon me **ct_ (count)** **nuk duhet të enkodohet.**

5. 3. Implementimi i Val Set

5. 3. 1. Set Val i balancuar sipas klasave

Kodi i mëposhtëm krijon një set të ri **val_df** duke përdorur 20% të data nga train. **Stratify** siguron që seti val të ruajë balancën midis klasave të sulmit duke përdorur të gjitha kategoritë prezentë në **data_train**.

```
train_df, val_df = train_test_split(data_train, test_size=0.2, random_state=42,
stratify=data_train['attack_cat'])
```

random_state=42 siguron që ndarja mes train dhe val është e njëjtë në çdo instancë, gjë që e bën më të lehtë evaluimin e performancës së modelit. E rëndësishme është mos të aplikohet **.fit()** mbi setin val, pasi duhet të trajtohet si të dhëna të panjohura.

```
x_train = preprocessor.fit_transform(train_df)
x_val    = preprocessor.transform(val_df)
x_test   = preprocessor.transform(data_test)

y_train = LabelEnc.fit_transform(train_df['attack_cat'])
y_val    = LabelEnc.transform(val_df['attack_cat'])
y_test   = LabelEnc.transform(data_test['attack_cat'])
```

5. 3. 2. Reduktimit të LR dhe EarlyStopping

Janë mekanizma “**callback**”, që ndërhyjnë mbi modelin gjatë procesit të trajnimit duke ndryshuar parametrat. Funksion kryesor kanë shmangien e Overfitting, kursim kohe, optimizim performance. Deklarimi i funksioneve:

```
tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
min_lr=1e-5)

tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=8,
restore_best_weights=True)
```


ReduceLROnPlateau përdoret për të ulur LR kur val_loss nuk përmirësohet për disa epoka rresht. Parametri factor=0.5 përgjysmon LR kur plotësohet kushti i mësipërm. Kjo lejon modelin që të marri hapa më të vegjël gjatë optimizimit të gradientit, me koston e një procesi më të gjatë trajnimi.

EarlyStopping përdoret për të ndaluar trajnimin automatikisht nëse nuk ka më përmirësim në val_loss për një numër x epokash specifikuar nga parametri *patience* = x. Opsioni **restore_best_weights=True** kthen në fund të trajnimit peshat më të mira të modelit (ato që patën val_loss më të ulët).

5.3.3. Interpretimi i Modelit me Val Set

Efekti i **stratify**, **ReduceLROnPlateau** dhe **EarlyStopping** mund të vëzhgohet në formën e grafikëve të Loss dhe Saktësisë. Tani ato shfaqin përmirësime të dukshme përgjatë epokave. Vlerat e regresionit për saktësinë e Val (të kuqe) janë pak më të vogla se ato të Train (blu). Ky është rezultati i pritshëm në modele ML.

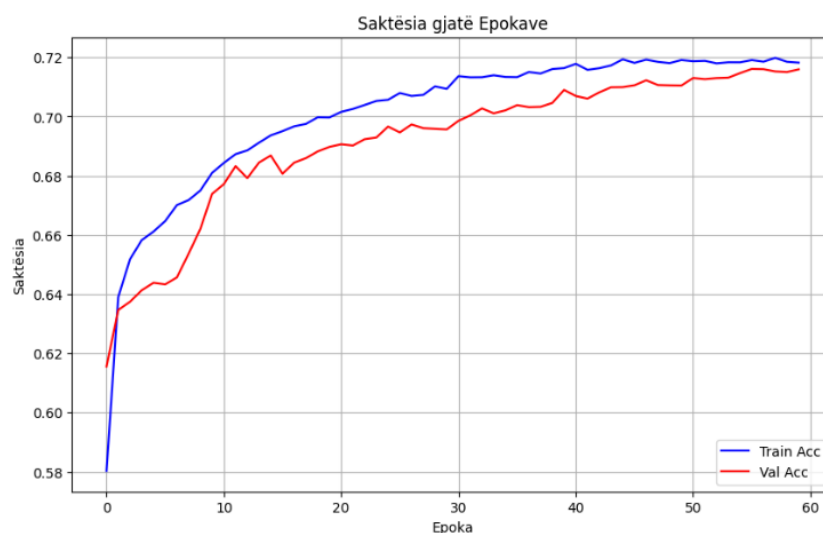


Figura 5-7 Saktësia e modelit Val përgjatë epokave

Vlera më e ulët e **Loss** për setin e validimit tregon që shtresa **Dropout()** është duke punuar në mënyrë korrekte, forcon modelin që të trajnohet më thellë. Gjithsesi shohim që për të dy setet shpejtësia e uljes së vlerës **Loss** afrohet me 0 rreth epokë 40-të. Modeli i thjeshtë po arrin limitin e kapacitetit të të mësuarit.

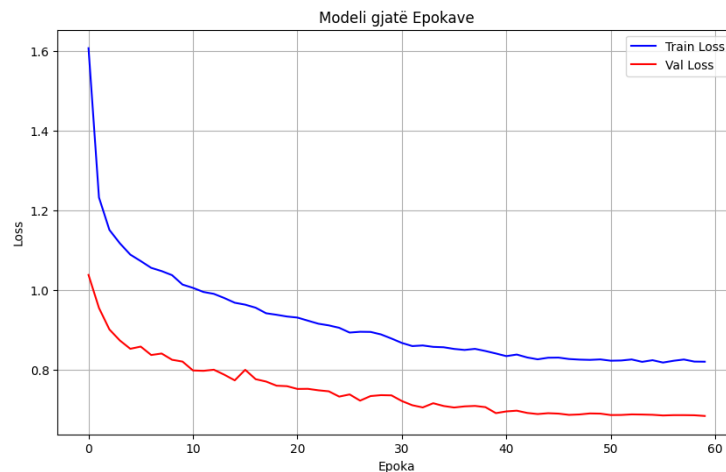


Figura 5-8 Loss e modelit Val përgjatë epokave

Analysis pati një përmirsim të ndjeshëm në recall $0.07 \rightarrow 0.23$. Duket qartë që optimizimet e reja kanë ndihmuar në kapjen e kësaj klase. Kemi një rënie të recall për klasat e Backdoor $0.71 \rightarrow 0.59$ dhe Dos $0.21 \rightarrow 0.15$, pjesa tjetër e modelit performon afërsisht me atë para OHE. Gjithsesi modeli vazhdon të vuajë nga precision i ulët, sidomos për klasa minoritare me më pak se 1000 hyrje në support (*Analysis, Backdoor, Worms, Shellcode*).

Modeli po keq-klasifikon rreth 25% të trafikut për Dos, Backdoor dhe rreth 5.8% të trafikut normal duke e ulur vlerën e precizionit në vetëm 3%.

Këto probleme rezultojnë për shkak të peshave lineare të klasave që u përmendën mbi sipër. Modeli është shumë i fokusuar në gjetjen e klasave minoritare të cilat shkaktojnë shumë FP, kryesisht duke u ngatërruar me klasa të tjera minoritare. Klasat mazhorante janë të sigurta me vlera të larta precisioni *Normal* dhe *Generic* $\rightarrow 1$, *Exploits* $\rightarrow 0.85$, *Reconnaissance* $\rightarrow 0.6$

Implementimi i OHE uli gjithashtu FP për **Backdoors** për klasa minoritare. **Analysis** 73.1% → 55.1%, **DoS** 56.4% → 42.2%, **Exploits** 23.8% → 17.7%.

Kategoria	precision	recall	f1 score	support
Analysis	0.03	0.23	0.06	677
Backdoor	0.06	0.59	0.11	583
DoS	0.35	0.15	0.21	4089
Exploits	0.85	0.47	0.61	11132
Fuzzers	0.26	0.7	0.38	6062
Generic	1	0.96	0.98	18871
Normal	1	0.58	0.73	37000
Reconnaissance	0.6	0.77	0.67	3496
Shellcode	0.11	0.72	0.2	378
Worms	0.02	0.8	0.04	44
accuracy				0.65 82332
macro avg	0.43	0.6	0.4	82332
weighted avg	0.86	0.65	0.71	82332

Figura 5-9 Performanca e modelit Val

5. 3. 4. Diferencat midis instancave për të njëjtin Model

Tabela më poshtë tregon rezultatet e performancës së **modelit Val Set** në 2 instanca të ndryshme, kodi është i njëjtë. Duket qartë diferenca mes rrezultateve të instancës së parë me të dytën, kjo e fundit duke ofruar Recall goxha më të lartë për *Analysis*, *Shellcode* dhe *Worms* (Fushat blu).

Kategoria	Modeli pa Seeding			Modeli me Seeding			Support
	precision	recall	f1 score	precision	recall	f1 score	
Analysis	0.03	0.23	0.06	0.05	0.3	0.08	677
Backdoor	0.06	0.59	0.11	0.05	0.53	0.09	583
DoS	0.35	0.15	0.21	0.38	0.13	0.19	4089
Exploits	0.85	0.47	0.61	0.85	0.49	0.62	11132
Fuzzers	0.26	0.7	0.38	0.26	0.68	0.38	6062
Generic	1	0.96	0.98	1	0.96	0.98	18871
Normal	1	0.58	0.73	1	0.58	0.73	37000
Reconnaissance	0.6	0.77	0.67	0.63	0.83	0.72	3496
Shellcode	0.11	0.72	0.2	0.12	0.83	0.21	378
Worms	0.02	0.8	0.04	0.03	0.8	0.06	44

Tabela 5-3 Krahësim mes modelit me dhe pa seeding

Normalisht ky ndryshim performace midis instancave shfaq probleme. Një model mund të jetë objektivisht më i mirë por mund të performojë më keq për shkak të rastësisë prezentë në ndarjen e seteve dhe operacioneve që përdoren gjatë trajnimit.

5. 3. 5. Implementimi i Determinizmit

Libraria keras ofron funksione që ruajnë konsistencën e operacioneve përgjatë instancave të ndryshme, duke siguruar të njëjtin rezultat për modelin.

```
tf.keras.utils.set_random_seed(42)
os.environ["TF_DETERMINISTIC_OPS"] = "1"
```

- `tf.keras.utils.set_random_seed(42)` trajton operacionet e ekzekutuara në cpu si ndarja e setit train në train + val.

- `os.environ ["TF_DETERMINISTIC_OPS"] = "1"` konverton disa prej operacioneve te GPU që janë në natyrë të rastësishme në deterministike.

Instanca e 2-të që jepte rezultate më të mira ka **seed 42**. Çdo test i paraqitur pas kësaj pike do të ketë këtë seed.

5. 4. Implementimi i transformimit LOG dhe Binarizimi i veçorive

5. 4. 1. Asimetria (Skew)

Skewness (asimetria) përshkruan sa shumë shpërndarja e të dhënave largohet nga simetria e një shpërndarjeje normale. Një shpërndarje normale (**pavarësisht nga vlera e mesatares μ apo devijimit standard σ**) ka **skew = 0**, pra është plotësisht simetrike. **Dataset-et e trafikut**

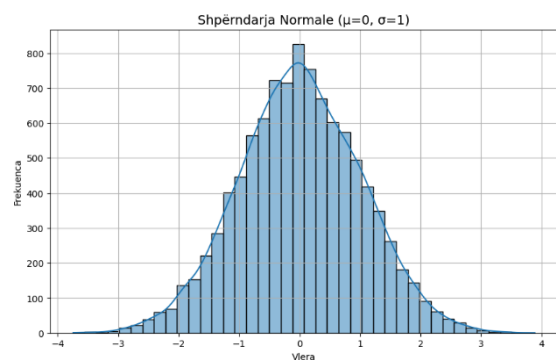


Figura 5-10 Shpërndarja Normale

të rrjetit zakonisht dominohen nga veçori me asimetri të lartë: shumica e trafikut është e sigurt dhe e standardizuar, ndërsa sulmet janë të rralla dhe shpesh shfaqen si outliers (vlera ekstreme shumë larg mesatares).

- **Normal (Skew ≈ 0):** formë simetrike
- **Asimetri Djathtas (Skew > 0):** shumica e vlerave janë të vogla, por ka disa shumë të mëdha, bishti shtrihet nga ana e djathtë.
- **Asimetri Majtas (Skew < 0):** shumica e vlerave janë të mëdha, por ka disa shumë të vogla, bishti i shtrihet nga ana e majtë.

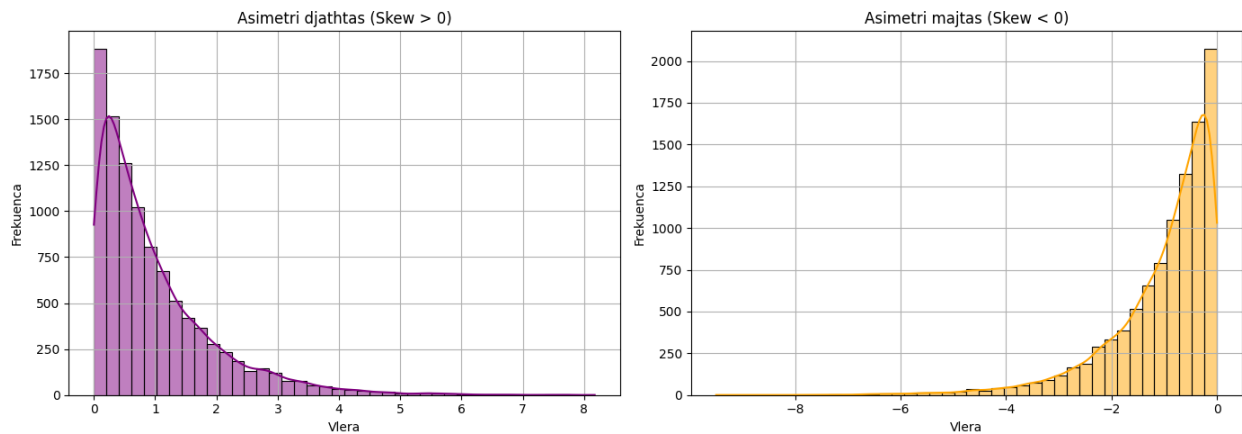


Figura 5-11 Paraqitja e vlerave të ndryshme të skew

Në modelet ML është praktikë e mirë të aplikohet një formë transformimi për të stabilizuar shpërndarjen e të dhënave kur skew është e madhe. Modeli mund të fokusohet në ekstremitetet duke çuar në luhajtje të gradientëve dhe rënie të ndjeshme performance. Si rregull transformimet kryhen kur:

- $|skew| < 0.5 \rightarrow$ shpërndarja afërsisht simetrike, nuk ka nevojë për transformim.
- $0.5 \leq |skew| < 1 \rightarrow$ asimetri mesatare, mund të aplikohet transformim.
- $|skew| \geq 1 \rightarrow$ asimetri e madhe, zakonisht aplikohet transformim.

Vlerat me skew shumë të lartë (mbi 10), mund të kenë nevojë për hapa më të veçantë procesimi. Zakonisht kjo lloj asimetrie rezulton për shkak të dominimit të madh të një kategorie në një veçori që mund të trajtohet me anë të transformimeve log. Funkzioni poshtë jep vlerat e `skew()` për kolonat numerike.

```
kol_skew_vlera = data_train[kol_num].skew().sort_values(ascending=False)
```

Kolona	Skew
trans_depth	167.336
response_body_len	76.34
sbytes	45.303
sloss	44.754
dloss	41.38
spkts	40.218
dbytes	39.761
dpkts	36.764
dinpkt	29.68
djit	29.544
ct_flw_http_mthd	23.451
sjit	17.652
ct_ftp_cmd	9.943

Kolona	Skew
is_ftp_login	9.943
sload	8.703
sinpkt	7.918
is_sm_ips_ports	7.778
dur	7.496
synack	5.786
ackdat	5.616
dload	4.691
tcprrt	4.038
smean	3.727
rate	3.319
dmean	2.826
ct_src_dport_ltm	2.171

Kolona	Skew
ct_dst_ltm	2.05
ct_src_ltm	1.842
ct_dst_sport_ltm	1.696
ct_dst_src_ltm	1.555
ct_srv_dst	1.529
ct_srv_src	1.521
stcpb	1.081
dtcpb	1.081
ct_state_ttl	0.928
dttl	0.895
dwin	0.197
swin	0.177
sttl	-0.678

Tabela 5-4 Vlerat e skew për veçoritë numerike, duke zbritur

Pjesa më e madhe e veçorive numerike kanë nevojë për transformim. 35 prej tyre kanë *skew* >

1. *ct_state_ttl*, *dttl*, *dwin*, *swin* nuk kanë nevojë të ndryshohen. *sstl* është e vetmja veçori me skew negative, por vlera absolute nuk është e mjaftueshme për të justifikuar një transformim.

5. 4. 2. Efekti i transformimit LOG në veçori

Në dataset-in **UNSW-NB15**, shumica e veçorive numerike janë **numërime (counts)** që kanë shpërndarje shumë të anuar: shumë vlera janë zero ose shumë të vogla, ndërsa pak raste kanë vlera shumë të mëdha. Kjo gjeneron outliers (vlera ekstreme) që ndikojnë në model. **Log transform (log1p)** përdoret shpesh sepse e “ngjesh” gamën e vlerave të larta dhe e bën shpërndarjen më të balancuar, duke ulur ndikimin e vlerave ekstreme dhe duke e bërë modelin më stabil. Ky transformim është i thjeshtë, i shpejtë dhe përshtatet natyrshëm me të dhënat jo-negative të UNSW-NB15.

```
log1p = FunctionTransformer(np.log1p, validate=False, feature_names_out="one-to-one")
num_log_pipeline = Pipeline([("log1p", log1p), ("scale", StandardScaler())])
transformers=[
    ("num_log", num_log_pipeline, kol_skew_trans)
```

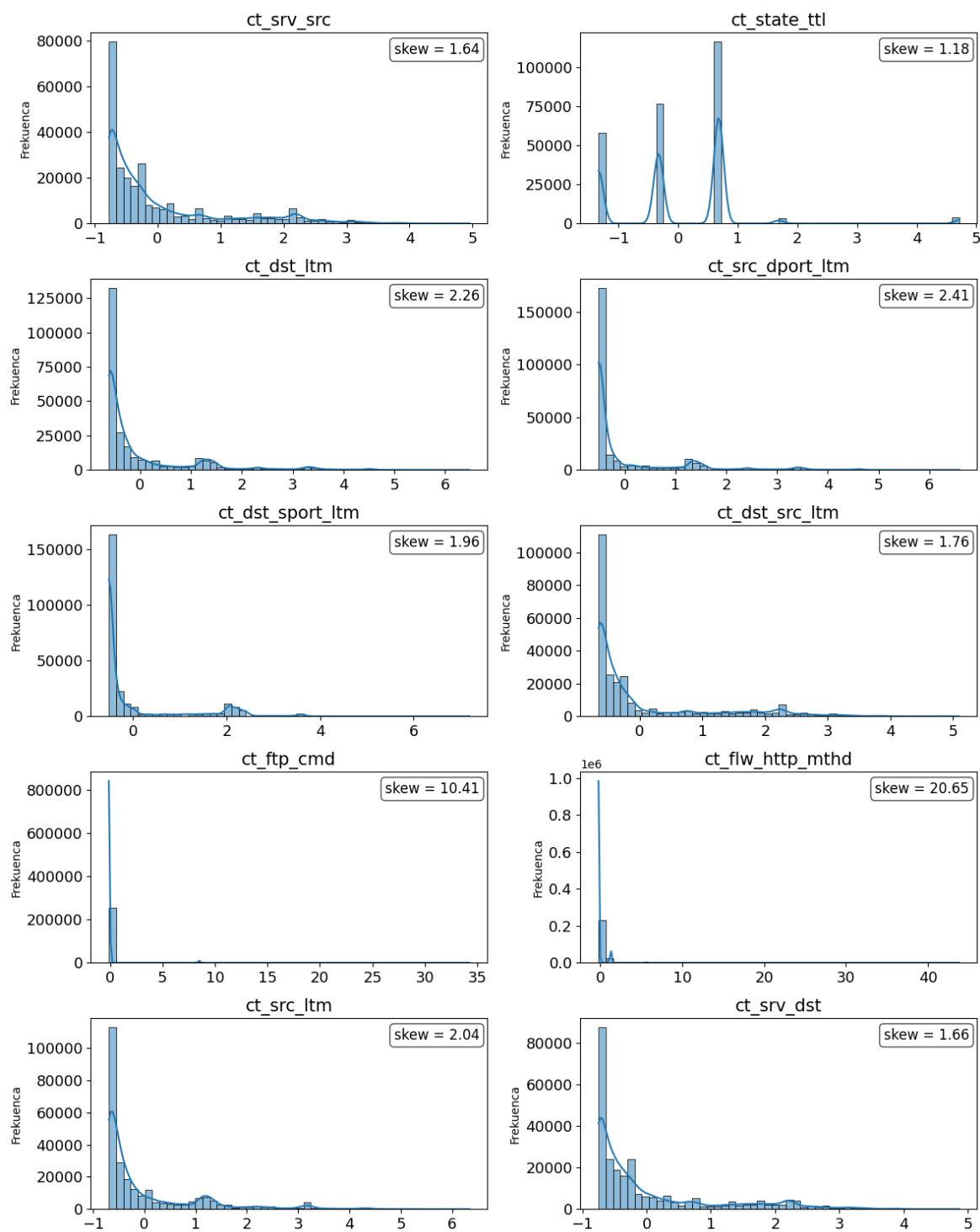


Figura 5-12 Veçoritë ct_ para transformimit LOG*

Transformimet log kanë ulur me sukses vlerën e skew përgjatë veçorive. Ndikimi më i madh dallohet te **ct_flw_htt_mthd** nga **20.65** → **4.16**. Gjithsesi ekzistojnë dhe kolona që nuk janë ndikuar shumë si **ct_ftp_cmd** **10.41** → **8.98**.

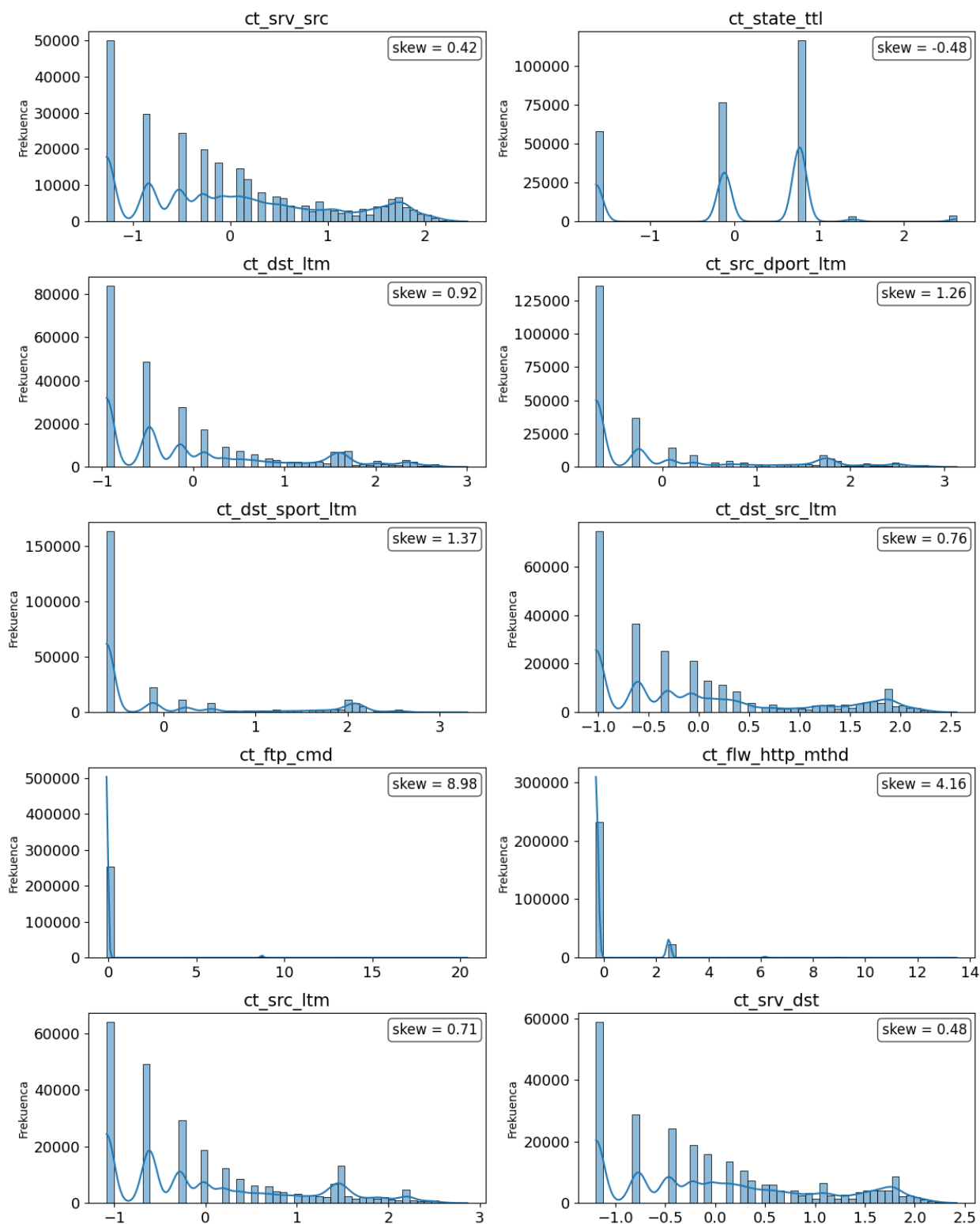


Figura 5-13 Veçoritë ct_* pas transformimit LOG

5. 4. 3. Trajtimi i kolonave Binare

Dy veçoritë me frekuencë më të ulët të vlerave unike **is_sm_ips_ports** dhe **is_ftp_login** janë binar (Moustafa, 2017) sipas dokumentimit të veçorive. Një gabim është bërë gjatë përgatitjes së dataset-it dhe **is_ftp_login** përmban 2 vlera shtesë.

```
array([0, 1, 2, 4])
```

Sipas dokumentimit 1 tregon që sesioni është aksesuar nga një user dhe password, në të kundërt 0. Zëvendësohen vlerat 2 dhe 4 me 1. (Maji, 2020)

```
data_num.loc[:, 'is_ftp_login'] = np.where(data_num['is_ftp_login'] > 1, 1,  
data_num['is_ftp_login'])  
data_num['is_ftp_login'].value_counts()
```

5. 4. 4. Interpretim i rezultateve për modelin Log dhe Binar

Kategoria	Modeli Para Trans			Modeli Pas Trans			Support
	precision	recall	f1 score	precision	recall	f1 score	
Analysis	0.05	0.3	0.08	0.04	0.26	0.07	677
Backdoor	0.05	0.53	0.09	0.05	0.55	0.09	583
DoS	0.38	0.13	0.19	0.32	0.12	0.17	4089
Exploits	0.85	0.49	0.62	0.83	0.51	0.63	11132
Fuzzers	0.26	0.68	0.38	0.25	0.66	0.36	6062
Generic	1	0.96	0.98	1	0.96	0.98	18871
Normal	1	0.58	0.73	1	0.58	0.73	37000
Reconnaissance	0.63	0.83	0.72	0.75	0.84	0.79	3496
Shellcode	0.12	0.83	0.21	0.14	0.89	0.24	378
Worms	0.03	0.8	0.06	0.03	0.84	0.05	44
accuracy	0.65			0.65			82332
macro avg	0.44	0.61	0.41	0.44	0.62	0.41	82332
weighted avg	0.86	0.65	0.71	0.86	0.65	0.71	82332

Tabela 5-5 Performanca e modelit para dhe pas transformimeve

Në pamje të parë modeli nuk është ndikuar nga transformimet log dhe trajtimi i duhur i klasave binare. Weighted avg (saktësia duke llogaritur peshat). Është e njëjtë për të dyja modelet.

Kemi përmirësim të recall të klasave minoritare të sulmeve, gjë e pritshme sepse transformimet log balancojnë shpërndarjen e të dhënave.

Backdoor	0.53	→	0.55	Klasat dhe <i>Analysis</i> , <i>DoS</i> , <i>Fuzzers</i> në të kundërt shfaqin ulje. Precizioni i ulët për klasa minoritare është akoma prezente.
Exploits	0.49	→	0.51	
Reconnaissance	0.83	→	0.84	
Shellcode	0.83	→	0.89	
Worms	0.8	→	0.84	

Performanca në klasat minoritare

Rezultatet tregojnë se klasat minoritare si *Analysis*, *Backdoor*, *Shellcode* dhe *Worms* vuajnë nga **precision shumë i ulët** (0.03–0.14). Shkaqet kryesore janë:

- **Numri i ulët i monstrave për testim** (p.sh. *Worms* ka vetëm 44 raste, *Shellcode* 378 raste). 52.7% (357) e sulmeve *Analysis* klasifikohen si *Backdoor* dhe 12.6% (85) si *Fuzzers*.
- **Mbivendosja e tipareve** me klasa më të mëdha si *DoS* dhe *Exploits*, duke bërë që modeli shpesh të ngatërrojë këto sulme të rralla me trafik më të zakonshëm.
- **Ndikimi i klasave të mëdha si Normal dhe Generic** ul më tej precizionin. 5.8% (2146 raste) e 37000 hyrjeve të trafikut *Normal* klasifikohen si *Analysis* që ka vetëm 677 raste në total, si rezultatat shkatërrohet precizionin për *Analysis*.
- **Recall i lartë, por precision i ulët** tregon se sistemi arrin të kapë shumicën e rasteve reale të këtyre klasave (p.sh. *Worms* → 84.1% recall), por një pjesë e madhe e parashikimeve rezultojnë në FP, çka ul precizionin. Siç është përmendur më përpara ky është një problem në dataset-e të pabalancuara.

Klasat kryesore të sulmeve

Për klasat e sulmeve më të mëdha si *Exploits* (11,132 raste) dhe *Fuzzers* (6,062 raste), precizioni dhe recall janë më të shëndetëshëm, respektivisht 0.83 dhe 0.25. Përfitojnë nga një volum më i madh të dhënash gjatë trajnimit, çka e bën modelin të gjenerojë parashikime më të qëndrueshme. Megjithatë dallohet një **trade-off mes precision dhe recall**: *Fuzzers* ka recall 65.8% por precision vetëm 25%. Analizimi i klasës *Normal* tregon se 31.3% (11655 raste, rreth dyfishi i rasteve aktualisht *Fuzzers*) të kësaj klase modeli e klasifikon si *Fuzzers*. Nurmi i lartë i FP ul precision.

Klasat dominante

Klasat **Generic** (18,871 raste) dhe **Normal** (37,000 raste) dominojnë dataset-in dhe kanë precision perfekt (1.0 për të dyja). Një analizë e **confusion matrix** tregon se:

- *Normal* ka **recall 0.58**, një pjesë e mirë e trafikut të shëndetshëm keq-klasifikohet si i dëmshëm.
- Kjo ul besueshmërinë e modelit, ka një numër të lartë FP (alarmesh fallco) **ndaj trafikut normal**. Çdo vëzhgues do të mbingarkohej nga ky trafik.
- Nga ana tjetër, *Generic* ruan një **balancë më të mirë mes precision(1) dhe recall (0.96)**, duke treguar që modeli e ka të lehtë të dallojë midis sulmeve të thjeshta dhe atyre më të specializuara.

Lidhja mes Normal dhe Sulmeve

Nga matrica shihet qartë që një pjesë e madhe e gabimeve ndodhin pikërisht mes **Normal dhe klasave të sulmeve**. Për shembull:

- Rastet reale të *Normal* shpesh klasifikohen si *Analysis* (5.8%) ose *Reconnaissance* (2.5%).

- Në të kundërt, disa sulme si **Backdoor** apo **Analysis** shpesh kategorizohen si **Normal** për shkak të mbivendosjes së tipareve të trafikut.

Ky fenomen është i rrezikshëm në aplikime praktike, sepse sjell **FP** (kur trafik normal raportohet si sulm) ashtu edhe **FN** (kur një sulm i vërtetë klasifikohet si normal). Ky problem do të trajtohet më poshtë.

Confusion Matrix (Diagonalja tregon recall)

Vlera Reale	Analysis	26.1%	52.7%	12.6%	3.0%	0.0%	0.0%	5.6%	0.0%	0.0%	0.0%
	Backdoor	26.9%	54.9%	13.9%	1.2%	0.7%	0.0%	0.0%	0.3%	1.5%	0.5%
	DoS	14.3%	51.6%	11.6%	13.3%	2.2%	0.0%	0.2%	1.1%	2.9%	2.6%
	Exploits	6.9%	20.8%	4.6%	50.8%	3.5%	0.0%	0.1%	3.6%	3.5%	6.2%
	Fuzzers	5.5%	12.9%	2.6%	0.5%	65.8%	0.0%	0.2%	2.4%	8.1%	2.0%
	Generic	0.1%	0.2%	0.5%	1.8%	0.4%	96.2%	0.0%	0.1%	0.2%	0.6%
	Normal	5.8%	0.3%	0.0%	0.5%	31.3%	0.0%	58.0%	1.0%	2.5%	0.5%
	Reconnaissance	1.5%	7.4%	0.9%	0.3%	0.5%	0.0%	0.0%	84.2%	3.4%	1.8%
	Shellcode	0.0%	0.5%	0.0%	0.0%	1.9%	0.0%	0.0%	6.6%	89.4%	1.6%
	Worms	0.0%	0.0%	0.0%	0.0%	4.5%	0.0%	0.0%	0.0%	11.4%	84.1%
		Vlera e Parashikuar									

Figura 5-14 Matrica e modelit pas Transformimeve

5. 5. Implementimi i Modelit me 2 Faza:

Problem kryesor janë numri i lartë i FP për sulmet që rezultojnë nga recall-i i ulët për klasën **Normal**, trafiku i sigurt klasifikohet si i dëmshëm. Për të trajtuar këtë problem do zhvillohet një model më kompleks që përforcon:

- Faza 1 (parashikim binar):** Klasifikon nëse data dikton trafik Normal apo një Sulm.

- **Faza 2 (parashikim kategorik):** Parashikon kategorinë e sulmit për rastet që u klasifikuan si sulm në fazën e parë.

Modeli i organizuar ka strukturë hierarkike, faza e parë diferencon sulmet nga aktiviteti Normal, faza 2 merr rastet e klasifikuara si sulme, dhe i klasifikon sipas kategorive.

Përpunimi i të dhënave do ndjekë hapat e modelit të mëparshëm për të 2 fazat.

1. Ndarja e kolonave – numerike, kategorike, binare
2. Transformimi i veçorive me asimetri të lartë
3. Enkodimi i veçorive kategorike
4. Shkallëzimi i të dhënave numerike

Përfitimet e modelit me 2 faza:

- **Ndarja e detyrave:** Dallimi mes trafikut *Normal* dhe *Anormal* është një detyrë në praktikë e ndryshme nga klasifikimi i llojit të sulmit.
- **Kontroll më i rreptë:** mund të ndryshohen parametra të një faze pa prekur fazën tjetër.
- **Ndjeshmëri ndaj klasave të rralla:** *class_weights* gjenerohen vetëm mbi të dhënat nga seti *Attacks*, mundëson daljen më në pah të tipareve specifike të çdo sulmi.

Sado ky model hierarkik ka një dobësi kritike. Gabimet e klasifikimit logjistik që realizohen në fazën e parë (trafiku normal i klasifikuar si sulm ose sulmet e klasifikuara si trafik normal) do të propagojnë nëpër fazën e dytë të trajnimit duke ndikuar padyshim në performancë. Për këtë arsye është kritik optimizimi i performancës së fazës së parë.

- FP – *Normal* klasifikohet si *Attack*
- FN – *Attacks* klasifikohet si *Normal*

5. 5. 1. Faza-1 Analiza Binare

Fillimisht duhet të konvertohen të gjitha kategoritë e *attack_cat* në standardin binar, që realizohet me funksionin e mëposhtëm. Për kategori të ndryshme nga *Normal* funksioni kthen

1. *Normal* → 0, 1 → *Attack*.

```
def to_binary(y_names):  
    return (y_names != 'Normal').astype(int)  
  
y_train_bin = to_binary(train_df["attack_cat"].values)  
y_val_bin   = to_binary(val_df["attack_cat"].values)  
y_test_bin  = to_binary(data_test["attack_cat"].values)
```

```
inp1 = tf.keras.Input(shape=(x_train.shape[1],), name="vecori_bin")  
h = tf.keras.layers.Dense(128, activation="relu")(inp1)  
h = tf.keras.layers.Dropout(0.30)(h)  
h = tf.keras.layers.Dense(64, activation="relu")(h)  
out1 = tf.keras.layers.Dense(1, activation="sigmoid", name="bin")(h)  
model_bin = tf.keras.Model(inp1, out1)
```

Layer (type)	Output Shape	Param #
vecori_bin (InputLayer)	(None, 192)	0
dense (Dense)	(None, 128)	24,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
bin (Dense)	(None, 1)	65

Figura 5-15 Faza 1 e modelit

Struktura e fazës 1 ndjek të njëjtën logjikë si modeli bazë, por duke ndryshuar funksionin e aktivizimit për output. Për klasifikim binar përdoret një funksion **sigmoid** $\sigma(x) = \frac{1}{1+e^{-x}}$ i cili kthen një vlerë brenda bashkësisë (0 1).

Zgjidhet një vlerë pragu për të përfunduar klasifikimin me kusht:

- nëse $\sigma(x) \geq v.pragu \rightarrow \text{klasa 1}$
- nëse $\sigma(x) < v.pragu \rightarrow \text{klasa 0}$

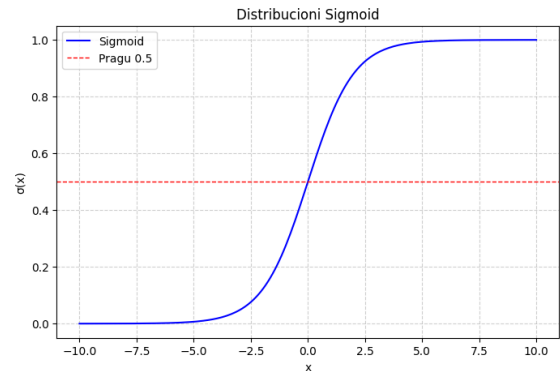


Figura 5-16 Funksioni Sigmoid

Modeli kryen optimizim të vazhdueshëm të vlerës së pragut, duke zgjedhur atë që maksimizon F1. Përdorimi i **class_weights** është efektiv meqë klasat janë më të balancuara.

- **Pesha 0: 0.361**
- **Pesha 1: 0.639**

Trajnimi i Fazës së parë ka zgjatur 45 epoka dhe ka mbaruar herët prej EarlyStopping sepse nuk kishte përmirësim në **val_loss** për 8 epoka. Data e filtruar nga ky model do të kaloi në [Faza-2].

```
Epoch 39/60
137/137 — 2s 14ms/step - accuracy: 0.9414 - loss: 0.1138 - val_accuracy: 0.9427 - val_loss: 0.1117 - learning_rate: 5.0000e-04
Epoch 40/60
134/137 — 0s 14ms/step - accuracy: 0.9409 - loss: 0.1142
Epoch 40: ReduceLRonPlateau reducing learning rate to 0.0002500000118743628.
137/137 — 2s 15ms/step - accuracy: 0.9416 - loss: 0.1135 - val_accuracy: 0.9425 - val_loss: 0.1117 - learning_rate: 5.0000e-04
Epoch 41/60
137/137 — 2s 15ms/step - accuracy: 0.9419 - loss: 0.1128 - val_accuracy: 0.9412 - val_loss: 0.1130 - learning_rate: 2.5000e-04
Epoch 42/60
137/137 — 2s 16ms/step - accuracy: 0.9420 - loss: 0.1123 - val_accuracy: 0.9414 - val_loss: 0.1126 - learning_rate: 2.5000e-04
Epoch 43/60
134/137 — 0s 14ms/step - accuracy: 0.9416 - loss: 0.1127
Epoch 43: ReduceLRonPlateau reducing learning rate to 0.0001250000059371814.
137/137 — 2s 15ms/step - accuracy: 0.9412 - loss: 0.1123 - val_accuracy: 0.9408 - val_loss: 0.1130 - learning_rate: 2.5000e-04
Epoch 44/60
137/137 — 2s 15ms/step - accuracy: 0.9418 - loss: 0.1121 - val_accuracy: 0.9412 - val_loss: 0.1131 - learning_rate: 1.2500e-04
Epoch 45/60
137/137 — 2s 15ms/step - accuracy: 0.9422 - loss: 0.1119 - val_accuracy: 0.9407 - val_loss: 0.1138 - learning_rate: 1.2500e-04
Epoch 45: early stopping
Restoring model weights from the end of the best epoch: 37.
2025-09-08 19:58:19.469963: E tensorflow/core/framework/node_def_util.cc:680] NodeDef mentions attribute use_unbounded_threadpool which is not in the op def
[Faza-1] Best threshold=0.390 | Objektivi=0.942 | Saktesia=0.942
```

Figura 5-17 Trajnimi i modelit gjatë fazës 1

5. 5. 2. Interpretimi i fazës 1

Ndarja në faza tregon një përmirësim të lartë të nivelit të recall për klasën *Normal*. Kërcimi është i ndjeshëm nga **0.58** – > **0.76**. Numri i FP (Trafik Normal që klasifikohet si sulm) është ulur por efekti është akoma i ndjeshëm. Ky error do të propagojë më tej në fazën e dytë të analizës.

Kategoria	[Faza-1] Normal vs Attack			Support
	precision	recall	f1 score	
Normal	0.98	0.76	0.86	37000
Attack	0.84	0.99	0.9	45332
accuracy			0.89	82332
macro avg	0.91	0.87	0.88	82332
Weight avg	0.9	0.89	0.88	82332

Tabela 5-6 Performanca për fazën e parë të modelit

Matrica tregon një pamje më të detajuar. Rreshti i parë tregon që **23.7%** e trafikut *Normal* është klasifikuar si *Sulm*. Gjithsesi vlera e recall për sulme është shumë e lartë. Vetëm 1.5% e sulmeve keq-klasifikohen. Numri i FP (Sulme që raportohen si trafik Normal) është i pranueshëm dhe tregon që optimizimet e aplikuara mbi model për recall të lartë janë efektive.

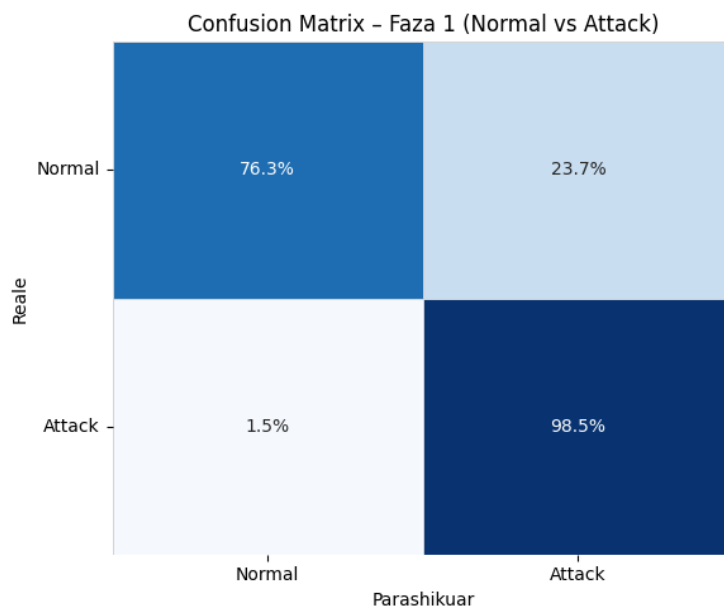


Figura 5-18 Matrica për fazën 1

5. 5. 3. Faza-2 Analiza Kategorike

Tani peshat e klasave do të llogariten bazuar vetëm në kategoritë e sulmit. Algoritmi është i njëjtë si modeli i parë duke ruajtur një lidhje lineare midis peshave, por tani ato do të jenë më pak të shpërndara për shkak se klasa Normal nuk përfshihet.

```
atk_weights = compute_class_weight("balanced", classes=atk_classes, y=y_tr_attack)
```

Analysis	: 6.630
Backdoor	: 7.593
DoS	: 1.081
Exploits	: 0.397
Fuzzers	: 0.729
Generic	: 0.332
Reconnaissance	: 1.264
Shellcode	: 11.709
Worms	: 102.000

Modeli për fazën 2 ndjek të njëjtën strukturë si ai bazik. Për klasifikimin e llojit të sulmit rritet numri i epokave të trajnimit. Me heqjen e klasës **Normal** modelit i lejohej që të mësoi më thellë pa u shqetësuar për keq-klasifikim e trafikut standard. Kushti i klasifikimit të data

është **argmax** (zgjidhet opsioni me probabilitet më të lartë). Gjithsesi, ekziston akoma risku për **overfitting** dhe mbahet algoritmi i EarlyStopping().

```
inp2 = tf.keras.Input(shape=(x_train.shape[1],), name="attack_kat_input")
h2 = tf.keras.layers.Dense(128, activation="relu")(inp2)
h2 = tf.keras.layers.Dropout(0.35)(h2)
h2 = tf.keras.layers.Dense(64, activation="relu")(h2)
out2 = tf.keras.layers.Dense(len(ATTACK_NAMES), activation="softmax",
name="attack_output")(h2)
model_atk = tf.keras.Model(inp2, out2)
```

Layer (type)	Output Shape	Param #
attack_kat_input (InputLayer)	(None, 192)	0
dense (Dense)	(None, 128)	24,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
attack_output (Dense)	(None, 9)	585

Figura 5-19 Faza 2 e modelit

```
94/94 — 1s 15ms/step - accuracy: 0.6905 - loss: 0.8361 - val_accuracy: 0.6826 - val_loss: 0.7770 - learning_rate: 1.0000e-05
Epoch 55/80
94/94 — 1s 15ms/step - accuracy: 0.6889 - loss: 0.8335 - val_accuracy: 0.6810 - val_loss: 0.7772 - learning_rate: 1.0000e-05
Epoch 56/80
94/94 — 2s 16ms/step - accuracy: 0.6894 - loss: 0.8293 - val_accuracy: 0.6815 - val_loss: 0.7767 - learning_rate: 1.0000e-05
Epoch 57/80
94/94 — 2s 16ms/step - accuracy: 0.6891 - loss: 0.8308 - val_accuracy: 0.6818 - val_loss: 0.7768 - learning_rate: 1.0000e-05
Epoch 58/80
94/94 — 2s 17ms/step - accuracy: 0.6887 - loss: 0.8345 - val_accuracy: 0.6818 - val_loss: 0.7772 - learning_rate: 1.0000e-05
Epoch 59/80
94/94 — 1s 16ms/step - accuracy: 0.6891 - loss: 0.8365 - val_accuracy: 0.6813 - val_loss: 0.7773 - learning_rate: 1.0000e-05
Epoch 60/80
94/94 — 2s 17ms/step - accuracy: 0.6889 - loss: 0.8298 - val_accuracy: 0.6810 - val_loss: 0.7773 - learning_rate: 1.0000e-05
Epoch 61/80
94/94 — 2s 16ms/step - accuracy: 0.6893 - loss: 0.8325 - val_accuracy: 0.6810 - val_loss: 0.7782 - learning_rate: 1.0000e-05
Epoch 62/80
94/94 — 2s 17ms/step - accuracy: 0.6885 - loss: 0.8309 - val_accuracy: 0.6809 - val_loss: 0.7776 - learning_rate: 1.0000e-05
Epoch 63/80
94/94 — 2s 16ms/step - accuracy: 0.6892 - loss: 0.8328 - val_accuracy: 0.6813 - val_loss: 0.7774 - learning_rate: 1.0000e-05
Epoch 64/80
94/94 — 2s 16ms/step - accuracy: 0.6887 - loss: 0.8343 - val_accuracy: 0.6814 - val_loss: 0.7773 - learning_rate: 1.0000e-05
Epoch 64: early stopping
```

Figura 5-20 Trajnimi i fazës 2 të modelit

5. 5. 4. Interpretimi i modelit me 2 faza

Rezultatet tregojnë se kalimi nga një model i thjeshtë **Log & Bin** drejt një arkitekture me 2 faza ka sjellë një përmirësim të dukshëm në balancimin e performancës midis klasave. Modeli me dy faza arrin të ruajë precizion të lartë për klasa të mëdha si *Normal* dhe *Generic*, ndërkohë që përmirëson ndjeshëm metrikat e klasave të vogla si *Analysis* dhe *Fuzzers*. Gjithashtu, f1-score për klasën Normal rritet nga 0.73 në 0.86, çka është një tregues i uljes së FP dhe i rritjes së besueshmërisë së modelit. Megjithatë, disa klasa si *Backdoor* dhe *DoS* mbeten sfiduese për t'u detektuar me saktësi të lartë.

Kategoria	Modeli Log & Bin			Modeli 2 Faza			Support
	precision	recall	f1 score	precision	recall	f1 score	
Analysis	0.04	0.26	0.07	0.08	0.5	0.14	677
Backdoor	0.05	0.55	0.09	0.03	0.3	0.06	583
DoS	0.32	0.12	0.17	0.37	0.15	0.22	4089
Exploits	0.83	0.51	0.63	0.83	0.5	0.62	11132
Fuzzers	0.25	0.66	0.36	0.34	0.56	0.42	6062
Generic	1	0.96	0.98	1	0.96	0.98	18871
Normal	1	0.58	0.73	0.98	0.76	0.86	37000
Reconnaissance	0.75	0.84	0.79	0.71	0.83	0.76	3496
Shellcode	0.14	0.89	0.24	0.14	0.85	0.24	378
Worms	0.03	0.84	0.05	0.02	0.84	0.05	44

Tabela 5-7 Krahasimi i modelit Log & Bin me atë me 2 faza

- *Analysis*: Modeli me dy faza arrin një përmirësim të konsiderueshëm të recall-it (nga 0.26 në 0.50), duke identifikuar gjysmën e rasteve të kësaj kategorie. Precizioni vazhdon të jetë akoma i ulët për shkak të numrit të ulët të monstrave prezentë në set.
- *Backdoor*: Kjo klasë shfaq humbje në performancë (f1-score nga 0.09 në 0.06). Ky rezultat sugjeron se edhe pas ndarjes në dy faza, modeli e ngatërron shpesh këtë kategori me klasa të tjera, kryesisht *Analysis*, gjë që shihet qartë edhe nga matrica e konfuzionit.

- **DoS:** Përmirësimi është i kufizuar (f1-score nga 0.17 në 0.22). Shumë raste të **DoS** shpërndahen ende në klasa si **Exploits** dhe **Reconnaissance**, duke treguar nevojën për veçori më dalluese në këtë kategori.
- **Exploits:** f1-score mbetet i ngjashëm (rreth 0.62), por balancimi i precision dhe recall është më i qëndrueshëm në modelin me dy faza. Sidoqoftë, ngatërrimi me **DoS** mbetet problem.
- **Fuzzers:** Rezulton një përmirësim i dukshëm (f1-score nga 0.36 në 0.42), kryesisht falë rritjes së precision-it, çka tregon se modeli arrin të reduktojë rastet e klasifikimeve të gabuara për këtë kategori.
- **Generic:** Për të dy modelet kjo kategori ka performancë shumë të lartë ($f1 \approx 0.98$). Ky rezultat sugjeron që karakteristikat e trafikut **Generic** janë më të dallueshme, duke e bërë klasifikimin më të lehtë.
- **Normal:** Përmirësimi është i madh, me rritje të f1-score nga 0.73 në 0.86. Ky rezultat është veçanërisht i rëndësishëm, pasi redukton numrin e FP (kalimi i trafikut normal si sulm) dhe rrit besueshmërinë e përgjithshme të sistemit.
- **Reconnaissance:** Performanca mbetet pothuaj e pandryshuar (f1-score nga 0.79 në 0.76). Edhe pse shumica e rasteve klasifikohen saktë (82.7%), ekzistojnë ende ngatërresa me **Exploits** dhe **DoS**.
- **Shellcode:** F1-score mbetet shumë i ulët (rreth 0.24), duke treguar se modeli ka vështirësi të dallojë këtë kategori, edhe pas ndarjes në dy faza. Dhe pse recall-i është i lartë (0.84) vlera e ulët e precisioni dëmton saktësinë.

- **Worms:** Performanca mbetet e kufizuar (f1-score ≈ 0.05). Shumë raste nga klasat e tjera paraqiten si Worm, fenomen që shkatërron precisionin. Numri i ulët i monstrave i jep një vlerë peshe shumë të lartë këtij sulmi dhe çon në një numër të lartë FP.

Faza-2 (Diagonalja tregon recall)

Vlera Reale	Analysis	49.9%	29.8%	11.7%	7.8%	0.3%	0.0%	0.1%	0.0%	0.3%	0.0%
	Backdoor	55.7%	29.7%	11.0%	0.5%	0.9%	0.0%	0.0%	0.3%	1.4%	0.5%
	DoS	17.1%	44.0%	15.3%	13.5%	2.5%	0.0%	0.3%	1.0%	2.9%	3.3%
	Exploits	9.6%	17.3%	5.5%	49.6%	3.4%	0.1%	0.5%	3.2%	3.4%	7.4%
	Fuzzers	11.1%	7.7%	2.3%	0.7%	55.6%	0.0%	9.7%	3.0%	7.9%	2.2%
	Generic	0.0%	0.2%	0.5%	1.7%	0.4%	96.2%	0.0%	0.0%	0.2%	0.7%
	Normal	2.4%	0.3%	0.0%	0.4%	16.3%	0.0%	76.3%	1.5%	2.3%	0.5%
	Reconnaissance	1.2%	6.9%	1.5%	0.2%	0.7%	0.0%	0.3%	82.7%	3.8%	2.7%
	Shellcode	0.0%	0.5%	0.0%	0.3%	1.9%	0.0%	0.3%	10.3%	84.9%	1.9%
	Worms	0.0%	0.0%	0.0%	0.0%	4.5%	0.0%	0.0%	0.0%	11.4%	84.1%
		Vlera e Parashikuar									

Figura 5-21 Matrica për modelin me 2 faza

Arkitektura me 2 faza sjell përfitime të dukshme në rritjen e saktësisë për disa klasa kritike si **Normal**, **Analysis** dhe **Fuzzers**, ndërkohë që ruan performancën shumë të lartë për **Generic**. Megjithatë, disa klasa si **Backdoor**, **DoS** dhe **Shellcode** mbeten problematike dhe kërkojnë teknika shtesë, si p.sh. oversampling të klasave minoritare, përzgjedhje dhe analizë më të mirë të veçorive, dhe implementimi i një modeli me 3 faza, e fundit e fokusuar në parashikimin e klasave minoritare.

KAPITULLI VI

KONKLUZIONE DHE REKOMANDIME

Në këtë temë u trajtua një nga problemet më kritike në sigurinë kibernetike, detektimi në kohë reale i sulmeve në një rrjet, duke përdorur koncepte nga **Machine Learning** për të zhvilluar një **IDS** bazuar mbi të dhëna nga dataset-i **UNSW-NB15**. Qëllimi kryesor ishte procesi i ndërtimit të një IDS në kushte individuale, i cili është testuar dhe përmirësuar në mënyrë graduale, duke nisur nga një model bazik dhe duke integruar optimizime të vazhdueshme që synojnë rritjen e saktësisë, ndjeshmërisë dhe balancimit midis klasave. Ky proces iterativ ka qenë i domosdoshëm për të kuptuar në mënyrë sistematike ndikimin që ka çdo teknikë mbi performancën përfundimtare të modelit.

Modeli u zhvillua nga një stad ku përdorte vetëm koncepte bazike nga supervised learning. Ky hap fillestar tregoi qartë kufizimet që paraqesin modelet ML, të cilat në përgjithësi rezultojnë nga vetë natyra e pabalancuar dhe diverse e të dhënave të hasura në rrjete. Kjo fazë tregoi si një IDS ndërtuar vetëm mbi koncepte bazike nga ML nuk është i mjaftueshëm për të ofruar rezultate të besueshme në një mjedis real.

Pas kësaj faze fillestare u morën një sërë hapash për përmirësimin e modelit:

- **Transformimet mbi të dhëna** (p.sh., log-transform për kolonat me skew të lartë) ndihmuan në reduktimin e ndikimit të vlerave ekstreme dhe përmirësuan stabilitetin e modelit.
- **Standardizimi**, hapi më i rëndësishëm që përdoret për të shkallëzuar të dhënat me metrika të ndryshme.

- **Enkodimi i veçorive kategorike** lejon modelin të përdorë në mënyrë më efektive të dhënat në forma jo numerike, duke e bërë modelin të aftë të mësojë nga variacionet midis protokolleve, shërbimeve dhe gjendjeve të komunikimit në rrjet.

Këto optimizime bazë sollën një rritje të dukshme të performancës, kryesisht në klasa mazhorante të sulmeve si *Exploits* dhe *Fuzzers*, por performanca e ulët në klasat minoritare si *Backdoor* dhe *Worms* mbetet e pranishme.

Rëndësia e teknikave të balancimit

Sfida kryesore ishte shpërndarja e pabalancuar e dataset-it, me klasën *Normal* që dominonte kundrejt gjithë dataset-it, ndërkohë klasa si *Worms/Backdoor* kanë rreth 100 raste. Për të adresuar këtë problem u aplikuan **class_weights**, të cilat i japin pesha më të larta klasave minoritare gjatë procesit të trajnimit. Kjo metodë rriti dukshëm recall-in për këto klasa, duke e bërë modelin më të ndjeshëm ndaj sulmeve të rralla të rrezikshme, por me kosto të lartë kundrejt precisionit të vetë klasave minoritare. Dhe pse në pamje të parë ky kompromis mund të duket i pranueshëm për një IDS, numri i lartë i FP, që rezultojnë nga recall i ulët për klasën *Normal* (0.58), do bëhen shpejt të pamenaxhueshme nga një ekip sigurie kibernetike.

Arkitektura me dy faza

Përmirësimi më i madh për modelin u vëzhgua me implementimin e arkitekturës me dy faza (Two-Stage Model). Faza e parë e modelit trajnohet për të dalluar trafikun normal nga trafiku i sulmeve në mënyrë sa më efektive, ndërsa faza e dytë, përshtatet për të klasifikuar llojin e sulmit duke përdorur një model të ngjashëm me atë bazik.

Rezultatet treguan qartë se kjo ndarje ishte efektive për performancën e përgjithshme të modelit:

- Në fazën e parë, **recall për trafikun normal u përmirësua ndjeshëm (0.58 → 0.72)**, duke reduktuar numrin e **FP**. Vetëm 1.5% e sulmeve u raportuan si **FN**, modeli po funksiononte mirë në ndalimin e sulmeve reale.
- Në fazën e dytë, klasat minoritare si *Analysis* dhe *Fuzzers* shfaqën një rritje të f1-score, pa kanibalizuar klasën dominante *Generic*.
- Për klasat më problematike si *Backdoor* dhe *Shellcode*, përmirësimi ishte minimal, modeli kapi më shumë raste por precisioni i ulët dëmton performancën.

Ky rezultat tregon se ndarja e problemit në disa faza redukton kompleksitetin dhe mundëson specializimin për lloje specifike sulmesh.

Sugjerime

Nga analizat e matricave të konfuzionit dhe raporteve të klasifikimit, doli qartë se modeli me dy faza është më i ekuilibruar. Ndërsa në qasjet bazike vetëm klasat dominante përfitonin nga saktësia e lartë, në modelin përfundimtar edhe klasat më të vogla arritën të marrin një performancë më të mirë. Kjo është veçanërisht e rëndësishme për aplikimet reale, ku një sulm i rrallë si **Worms** ose **Backdoor** mund të ketë pasoja katastrofike nëse nuk detektohet.

Ekzistojnë ende disa teknika optimizimi që mund të eksplorojnë për të rritur më tej efikasitetin e modelit. Një mundësi është përdorimi i teknikave të avancuara të balancimit të të dhënave, si oversampling si SMOTE (rritja artificiale e numrit të rasteve të vërteta për klasat minoritare) ose data augmentation,. Këto teknika rrisin ndjeshmërinë kundrejt këtyre klasave.

Po ashtu, përzgjedhja dhe reduktimi i dimensioneve të veçorive përmes metodave si PCA ose autoencoders do të ndihmonte në eliminimin e tipareve të panevojshme dhe reduktimin e kompleksitetit. Nga ana arkitekturore, mund të merren në konsideratë modele më të fuqishme si rrjetet e përsëritura (RNN, LSTM) ose edhe arkitekturat Transformer, të cilat kanë treguar

potencial të madh në analizimin e të dhënave sekuenciale dhe në detektimin e anomalive. Një tjetër drejtim për optimizim është përdorimi i metodave më të avancuara të tuning-ut të hiperparametrave si Bayesian Optimization ose Grid Search të kombinuar me *cross-validation*, për të rritur stabilitetin dhe përgjithësimin e modelit. Në fund, për përdorim praktik, modeli mund të zgjerohet me mekanizma të detektimit në kohë reale kundrejt sulmeve kundërshtare, duke e bërë më të sigurt dhe më të përshtatshëm për mjedise reale të sigurisë kibernetike.

BIBLIOGRAFIA

- Biggio, B., & Roli, F. (2018). Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition, 84*, 317–331. https://doi.org/10.1016/j.patcog.2018.07.023
- Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials, 18*(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras and tensorflow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Inc.
- Keshk, M., Moustafa, N., Sitnikova, E., & Creech, G. (2017). Privacy preservation intrusion detection technique for SCADA systems. *2017 Military Communications and Information Systems Conference (MilCIS)*, 1–6. IEEE. https://doi.org/10.1109/MilCIS.2017.7348942
- Maji, S. (2020, September 19). Building an intrusion detection system on UNSW-NB15 dataset based on machine learning algorithm. *Medium*. https://medium.com/@subrata.maji16/building-an-intrusion-detection-system-on-unswnb15-dataset-based-on-machine-learning-algorithm-16b1600996f5
- Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 dataset) [Data set]. Australian Centre for Cyber Security (ACCS).
- Moustafa, N., Creech, G., Slay, J., & Keshk, M. (2018). Generalized Outlier Gaussian Mixture technique based on Automated Association Features for Simulating and Detecting Web Application Attacks. *IEEE Transactions on Sustainable Computing*. https://doi.org/10.1109/TSUSC.2018.2808450
- Moustafa, N., Keshk, M., & Slay, J. (2018). A new threat intelligence scheme for safeguarding Industry 4.0 systems. *IEEE Access, 6*, 32910–32924. https://doi.org/10.1109/ACCESS.2018.2846602
- Moustafa, N., et al. (2018). Anomaly detection system using beta mixture models and outlier detection. In *Progress in Computing, Analytics and Networking* (pp. 125–

- 135). Springer, Singapore. [https://doi.org/10.1007/978-981-10-7871-2_12](https://doi.org/10.1007/978-981-10-7871-2_12)
- Moustafa, N., et al. (2018). Flow Aggregator Module for analysing network traffic. In **Progress in Computing, Analytics and Networking** (pp. 19–29). Springer, Singapore. [https://doi.org/10.1007/978-981-10-7871-2_2](https://doi.org/10.1007/978-981-10-7871-2_2)
 - Moustafa, N., et al. (2018). A network forensic scheme using correntropy-variation for attack detection. In **IFIP International Conference on Digital Forensics** (pp. 251–265). Springer, Cham. [https://doi.org/10.1007/978-3-319-99277-8_15](https://doi.org/10.1007/978-3-319-99277-8_15)
 - Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A deep learning approach to network intrusion detection. **IEEE Transactions on Emerging Topics in Computational Intelligence*, 2*(1), 41–50. <https://doi.org/10.1109/TETCI.2017.2772792>
 - Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. **2010 IEEE Symposium on Security and Privacy**, 305–316. <https://doi.org/10.1109/SP.2010.25>